DOT-Design

Version Development, 2026-10-21

Overview

DOT-Design defines the portable model and reference tooling used to describe, validate, deploy, and compose Dynamic Open Trust (DOT) components in the MARE 6G security ecosystem.

A DOT packages four concerns without collapsing them into one runtime-specific description:

- **capability** — Skills describe what the component does;
- **contract** — OpenAPI, AsyncAPI, Protobuf, or GraphQL documents describe authoritative interactions;
- **composition** — Anchors give selected Contract operations stable identities and `PROVIDES` or `REQUIRES` roles; and
- **deployment** — Runtimes and deployment manifests describe where and how the component can run.



`dot.json` is the deployable DOT definition and the only source inspected by `dotx deploy`. Connections between deployed DOT instances are described in a separate composition document. Concrete addresses, credentials, and live policy results never become Anchor fields.

Documentation map

Read	When you need to
Getting started	Install the released CLI, deploy Prometheus with Docker Compose, verify its API, and clean up.
Contributor guide	Build the project, run shared local/GitHub checks, and generate documentation.
Guided demo	Follow a five-minute presenter path or a deeper onboarding walkthrough from catalogue validation to live Contract evidence.
Architecture	Understand repository boundaries, sources of truth, runtime/deployment separation, and the composition resolution path.
DOT Anchors and Composition	Author Anchors, bind them to services, connect DOT instances, materialize configuration, handle security inputs, and operate a composition.
Verification and examples	Choose conformance, dry-run, source-adapted, or live evidence and find the executable MARE PoC scenarios.
DOT-Domain	Work with the Java aggregate, value objects, validation, Contracts, Runtimes, and deployment manifests.
Integration handoff	Implement a catalogue, composer, or SFC adapter without coupling to a specific service, language, framework, or storage technology.

Repository components

Component	Responsibility	Build ecosystem
`DOT-Domain`	Java domain model and validation library. It is not a web application.	Maven 3.10.0 / Java 27
`DOT-CLI`	Independent `dotx` CLI for authoring support, validation, inspection, deployment, runtime lifecycle, and the provisional composition executor.	Go 1.27.2
`DOT-Repository`	Independently cloneable Git submodule containing reusable DOT definitions, the generated JSON Schema, and optional high-level Skill classifications.	Git submodule
`DOT-Testing`	Repository-level conformance, composition, interoperability, and end-to-end evidence. It is test infrastructure, not a Maven module.	Maven, Go, shell, container tools

Supported Contract standards

Standard	Anchor selector
OpenAPI	RFC 6901 URI-fragment JSON Pointer to an Operation Object
AsyncAPI	RFC 6901 URI-fragment JSON Pointer to an Operation Object
Protobuf	Fully qualified service method
GraphQL	Root-field schema coordinate

Contractless Anchors remain available for interactions that cannot be described by one of these standards. They must declare direction, protocol, and media type explicitly.

Maturity and scope

The DOT-owned Anchor model, service-port binding, generated schema, repository resolution, conformance fixtures, `dot.mare6g.eu/v1` composition document, and `dotx compose` executor form the stable 0.2.4 Composition v1 profile.

The reference executor is intentionally fail-closed. It does not claim to be a general multi-host orchestrator, trust-policy engine, active security scanner, or replacement for Kubernetes, Helm, Docker, Vagrant, or an SFC implementation. See the supported composition profile for the exact boundary.

First steps

```
# Domain unit tests; no MongoDB required
mvn -pl DOT-Domain test
```

```
# Install the pinned CLI checkout on PATH
task cli:install

# Validate and inspect the canonical composition example
composition=DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml
dotx compose validate "$composition"
dotx compose plan "$composition"

# Run the complete Anchor/composition repository gate
task testing:conformance
```

Continue with Getting started for prerequisites and command context, then use Guided demo for a reproducible end-to-end walkthrough.

Project links

- MARE project
- MARE proof-of-concept catalogue
- DOT-Design source repository

Getting started

Run your first DOT on Linux amd64 using the released CLI and the existing Prometheus definition. You need Bash, Git, GitHub CLI (`gh`), `curl`, `jq`, SHA-256 utilities, and Docker with Docker Compose. No Java, Go, Kubernetes, or Task is needed for this path.

This example runs Prometheus 3.15.0 on port `18909`. Allow time for initial image and Contract downloads. Use an isolated development host: the Compose manifest publishes unauthenticated HTTP on all host interfaces. Restrict access using your host firewall. Cleanup deletes the example's metrics data.

Install the released CLI

Authenticate with an account granted access to the private MARE repositories and releases. From a workspace directory:

```
gh auth login
gh auth setup-git
mkdir -p dot-quickstart-download
cd dot-quickstart-download
gh release download v0.2.4 --repo MARE-EU-Project/DOT-CLI \
  --pattern dotx-0.2.4-linux-amd64 --pattern SHA256SUMS
sha256sum --check --ignore-missing SHA256SUMS
mkdir -p "$HOME/.local/bin"
install -m 0755 dotx-0.2.4-linux-amd64 "$HOME/.local/bin/dotx"
export PATH="$HOME/.local/bin:$PATH"
dotx version
cd ..
```

Stop if checksum verification fails. Expected version: `dotx 0.2.4`. Add the `PATH` export to your shell profile for future sessions. The release also provides an SBOM and build/acceptance revision manifest.

Check out the catalogue

Only the independently cloneable catalogue is needed. Pin the revision used by this walkthrough, then check your Docker access:

```
gh repo clone MARE-EU-Project/DOT-Repository
cd DOT-Repository
git checkout --detach 49395d60a70a7f22d9109a3e5c4006c68f3218ad
docker info
docker compose version
```

Run the following blocks in the same Bash shell from the **DOT-Repository root**. Run each block only after the preceding block succeeds. The selected descriptor embeds its Compose manifest; no

neighbouring Compose file is inferred.

Select and validate

Choose an unused runtime name. For a different host port, run `export DOTX_HTTP_PORT=18919` before the selection block.

```
DOT_REPOSITORY="${DOT_REPOSITORY:-$PWD}"
DOT_BUNDLE="$DOT_REPOSITORY/DOTs/external/Prometheus-DOT"
DOT_NAME="${DOT_NAME:-dot-quickstart}"
DOTX_HTTP_PORT="${DOTX_HTTP_PORT:-18909}"
DOT_PROJECT_FILTER="label=com.docker.compose.project=$DOT_NAME"
export DOTX_HTTP_PORT
```

```
dotx validate "$DOT_BUNDLE" --schema "$DOT_REPOSITORY/Schema/schema.json" \
  --profile reusable
dotx inspect "$DOT_BUNDLE" --json
dotx deploy "$DOT_BUNDLE" --deployment docker-compose --name "$DOT_NAME" --dry-run
```

Validation checks the schema, official HTTPS OpenAPI Contract and SHA-256, Anchor selectors, and reusable deployment bindings. Inspection shows deployment choices. The exact ID `docker-compose` selects only that mechanism. Dry-run prints its commands; it does not prove that a service is running.

Deploy and verify

```
dotx deploy "$DOT_BUNDLE" --deployment docker-compose --name "$DOT_NAME" \
  --env "DOTX_HTTP_PORT=$DOTX_HTTP_PORT"
```

Wait up to 60 attempts (each HTTP attempt has a two-second timeout), then verify the service's build-information API and version:

```
ready=false
for ((attempt=0; attempt<60; attempt++)); do
  if curl --fail --silent --max-time 2 \
    "http://127.0.0.1:$DOTX_HTTP_PORT/-/ready"; then
    ready=true
    break
  fi
  sleep 1
done
test "$ready" = true
curl --fail --silent --show-error --max-time 5 \
  "http://127.0.0.1:$DOTX_HTTP_PORT/api/v1/status/buildinfo" |
  jq -e '.status == "success" and .data.version == "3.15.0"'
```

Expected: `Prometheus Server is Ready.`, followed by `true` from `jq`. Check runtime status:

```
dotx ps
```

Expect a running workload. Open `http://localhost:18909` (or your chosen port) to view Prometheus.

The `build-information` Anchor selects the official OpenAPI operation whose server prefix produces `/api/v1/status/buildinfo`. The `promql-query-api` Anchor selects query operations; both bind to the deployed service's `http` port. Its explicit contractless scrape-target requirement describes metrics ingestion; this example does not configure another DOT as a scrape target.

The pinned image's default configuration starts the service. The descriptor declares a Compose-managed `prometheus-data` volume. Data survives container restarts until the volume is deleted.

Clean up



For Compose, `dotx destroy` runs `down --volumes`: it deletes this instance's containers, network, and named volume, including collected metrics. Use the same `DOT_NAME` as deployment. Never target another user's instance.

```
dotx destroy "$DOT_BUNDLE" --deployment docker-compose --name "$DOT_NAME" &&
remaining="$(docker ps -aq --filter "$DOT_PROJECT_FILTER")" &&
test -z "$remaining" &&
remaining="$(docker network ls -q --filter "$DOT_PROJECT_FILTER")" &&
test -z "$remaining" &&
remaining="$(docker volume ls -q --filter "$DOT_PROJECT_FILTER")" &&
test -z "$remaining"
```

The final checks succeed silently. If interrupted, restore your original selection variables and repeat cleanup. Downloaded images remain cached.

Next: connect existing DOTs

For the existing Keycloak/APISIX/OAuth composition, use a full DOT-Design checkout with initialized submodules; see Contributor guide. From the **DOT-Design root**, with `dotx` on `PATH`:

```
composition=DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml
dotx compose lock "$composition" --check
dotx compose plan "$composition" --locked
dotx compose apply "$composition" --locked --dry-run
```

The three wires connect Keycloak discovery to APISIX, Keycloak's token endpoint to the OAuth client, and APISIX's protected operation to that client. The plan resolves endpoints and configuration. Live acceptance additionally proves anonymous rejection and authenticated access; the harness prepares Kubernetes, native Secrets, and workloads. Follow Guided demo and Verification and examples for that path.

To author a component, continue with Authoring Anchors. For development toolchains and local/GitHub CI gates, use Contributor guide.

Troubleshooting

Symptom	Action
Release or clone returns 404/403	Check `gh auth status` and organization repository access, including SSO authorization if required. Run `gh auth setup-git` for HTTPS cloning.
CLI missing or wrong version	Check `command -v dotx`; put `$HOME/.local/bin` first on `PATH`.
Docker permission denied	Check `docker info` as your user. Ask the administrator for the intended Docker access, then log in again. Docker group access grants host-level power.
Compose unavailable	Install the Compose plugin/package for your Docker distribution and verify `docker compose version`. The legacy `docker-compose` executable is insufficient.
Port already allocated	Clean up the failed example, select a free `DOTX_HTTP_PORT`, and deploy again with the same name and explicit `--env` override.
Contract download or digest check fails	Check network/proxy access to the declared URL. Never bypass its digest. Private raw GitHub URLs require `MARE_READ_ONLY_TOKEN` or `GITHUB_TOKEN`; `gh` login alone does not supply those to `dotx`. Use `--offline` only with matching verified local Contract assets.
Readiness or version fails	Inspect `docker compose -p "$DOT_NAME" logs --tail=100` and `dotx ps --all`, check the selected image/port, then clean up.
Interrupted cleanup	Restore the original name and catalogue path, repeat cleanup, and inspect resources carrying `com.docker.compose.project=$DOT_NAME`.

Download the shared command blocks. Contributors rehearse them from DOT-Design with `task testing:quickstart:dry-run` or the opt-in `task testing:quickstart:live`.

Guided demo

This guided demo turns the stable Composition v1 profile into a short, reproducible story. It starts with a real reusable external DOT, follows a Contract through Anchors and a composition wire, and finishes by rendering a multi-DOT lifecycle without changing a runtime.

The core path takes about five minutes on an existing checkout. It is read-only apart from temporary files created and removed by the CLI. The optional live path uses Docker and performs deployment, protocol probes, and cleanup.



Run the commands from the DOT-Design repository root. Keep evidence levels separate: a valid plan proves deterministic resolution, a dry-run proves that the executor can render supported runtime actions, and only a live probe proves the observed protocol exchange.

Choose a path

For a fully executable local presentation spanning Docker, Kubernetes, and OpenAPI/GraphQL/Protobuf/AsyncAPI wires, follow Docker/Kubernetes four-Contract composition. It includes the rehearsal, real probe outcomes, cleanup, and evidence boundaries.

Path	Time	Demonstrates	Runtime mutation
Presenter	About 5 minutes	Repository validation, a reusable external DOT, 21 cross-standard wires, and a Kubernetes lifecycle rendering	None
New developer	About 15 minutes	The presenter path plus selected plan details, authoritative Contract facts, endpoint materialization, and evidence boundaries	None
Live acceptance	Depends on image downloads and host capacity	GraphQL, all four finite gRPC shapes, and MQTT, NATS, Kafka, and AMQP 0-9-1 delivery against real producers and brokers	Disposable Docker resources

The story in one diagram

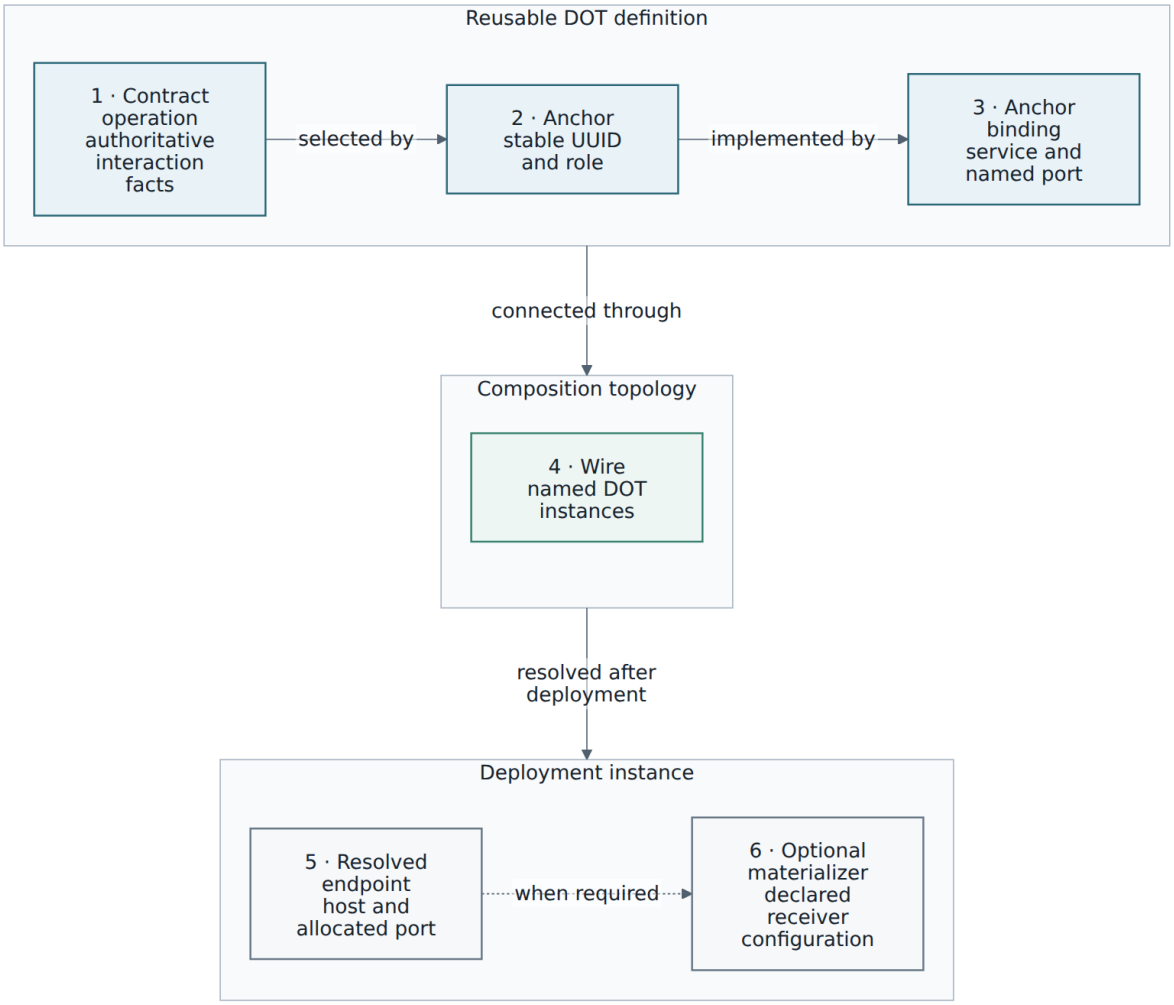


Figure 1. From an authoritative Contract operation to a deployed connection

The Contract owns protocol semantics. An Anchor gives one selected Contract operation a stable logical identity. A deployment-service binding identifies the service port that implements the Anchor. A composition wire connects two named DOT instances and may materialize the resolved endpoint into configuration already declared by the consumer.

Prepare the checkout

For a first deployment with the released CLI, follow Getting started. This presenter path uses a full checkout from Contributor guide with initialized `DOT-CLI` and `DOT-Repository` submodules. The demo needs `dotx`, `jq`, and network access when URL-backed Contracts are not already cached.

```
git submodule update --init --recursive
task cli:install
dotx version
```

For the version-aligned stable release, the last command reports:

```
dotx 0.2.4
```

The independently released Linux-amd64 binary, its SBOM, release manifest, and checksums are also available from the DOT-CLI v0.2.4 release. Use `sha256sum --check SHA256SUMS` before executing a downloaded release asset.

Stage 1: validate the authoritative catalogue

```
dotx validate DOT-Repository
```

This recursively discovers every `dot.json`; there is no separate inventory. The command validates schema shape, identities, Skills, Contract payload integrity, selectors, service ports, Anchor bindings, repository placement, and catalogue-wide uniqueness. Contract-backed external and internal bundles also receive the strict reusable profile.

What this proves

The checked-out catalogue is internally consistent and every declared reference resolves. It does not deploy a workload or claim that a third-party service is reachable.

Stage 2: inspect a reusable external DOT

Keycloak is a useful example because its canonical DOT uses official, digest-pinned Contracts while offering multiple deployment mechanisms.

```
dotx inspect DOT-Repository/DOTs/external/Keycloak-DOT --json |
  jq '{name, version,
       skills: (.skills | length),
       contracts: (.contracts | length),
       anchors: (.anchors | length),
       deployments: (.deployments | length),
       supportedDeploymentSystems}'
```

The `0.2.4` catalogue produces this compact projection:

```
{
  "name": "Keycloak-DOT",
  "version": "0.2.0",
  "skills": 2,
  "contracts": 3,
  "anchors": 4,
  "deployments": 4,
  "supportedDeploymentSystems": [
    "DOCKER_COMPOSE",
    "DOCKER_IMAGE",
    "HELM_CHART"
  ]
```

```
}
```

What this proves

One reusable DOT root can describe several supported deployment choices without forking its identity for each PoC. Deployment-instance realms, routes, credentials, and endpoints remain composition inputs rather than copied DOTs.

Stage 3: resolve the cross-standard composition

The canonical Contract matrix connects six DOT instances with 21 compatible wires. Its committed lock pins every DOT, Contract, configuration input, and portable resolved plan.

```
dotx compose lock \
  DOT-Testing/composition/contract-standards/scenario.yaml --check

dotx compose validate \
  DOT-Testing/composition/contract-standards/scenario.yaml --locked

dotx compose plan \
  DOT-Testing/composition/contract-standards/scenario.yaml --locked --json |
  jq '{apiVersion, name, version,
       dotInstances: (.dots | length),
       wires: (.wires | length),
       compatibleWires: ([.wires[] |
         select(.compatibility == "COMPATIBLE")] | length),
       standards: ([.wires[].contractStandard] | unique),
       interactions: ([.wires[].providerInteraction] | unique)}'
```

Expected summary:

```
{
  "apiVersion": "dot.mare6g.eu/v1",
  "name": "contract-standards-matrix",
  "version": "1.0.0",
  "dotInstances": 6,
  "wires": 21,
  "compatibleWires": 21,
  "standards": ["ASYNCAPI", "GRAPHQL", "OPENAPI", "PROTOBUF"],
  "interactions": [
    "BIDIRECTIONAL_STREAM", "CLIENT_STREAM", "MUTATION", "QUERY",
    "REQUEST_RESPONSE", "SEND", "SERVER_STREAM", "SUBSCRIPTION"
  ]
}
```

The project gate asserts the complete result rather than only printing a summary:

```
task testing:compose:contract-standards
```

What this proves

Every provider and consumer selector resolves against its authoritative Contract; all 21 pairs are compatible; endpoints, interactions, probes, and materializers can be derived without copying standard-specific semantics into Anchors.

Follow one AsyncAPI wire

Inspect the Kafka wire to see how standard-derived facts and deployment facts remain separate:

```
dotx compose plan \
  DOT-Testing/composition/contract-standards/scenario.yaml --locked --json |
  jq '.wires[] | select(.name == "asyncapi-kafka") |
      {name, compatibility, contractStandard, contractFormat,
       providerInteraction, consumerInteraction, channelAddress,
       connectionEndpoint, materializers, probe}'
```

The AsyncAPI Contract supplies the complementary send/receive operations, channel, message schema, Kafka binding, and correlation field. The composition selects the broker instance and materializes its resolved address and topic for the consumer. A plan still does not claim that a message was delivered.

Stage 4: render a lifecycle without mutating it

The APISIX, Keycloak, and OAuth-client scenario demonstrates dependency order, configuration materialization, native Kubernetes Secret references, rollout readiness, and an exposed Anchor:

```
dotx compose lock \
  DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml --check

dotx compose apply \
  DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml \
  --locked \
  --namespace dot-guided-demo \
  --dry-run \
  --forward protected-metadata-through-gateway=19080
```

The rendering ends with:

```
Dry run complete; composition state was not written.
```

The command may create bounded temporary rendering files and removes them when it exits. It does not create Kubernetes resources, start a port forward, or write composition state. The rendered order is Keycloak, APISIX, then the OAuth client, matching the declared readiness dependencies.

What this proves

The reference executor accepts the locked inputs and can derive the exact platform-native actions. It does not prove rollout success, credentials, or HTTP behavior; those require the live acceptance layer.

Stage 5: connect claims to evidence

Claim	Executable evidence	Boundary
Catalogue correctness	`dotx validate DOT-Repository`	Static and Contract resolution evidence; no deployment
Cross-standard interoperability	`task testing:compose:contract-standards`	21 deterministic compatible plans; no live traffic
Live Contract probes	`task testing:compose:contract-probes:live`	Real producer and brokers on disposable Docker resources
Runtime configuration portability	`task testing:compose:runtime-configuration`	Dry-run rendering across six adapters
Workload isolation	`task testing:compose:service-workloads:live`	Live repeated Compose, Kubernetes, and Helm instances
Portable-runtime isolation	`task testing:compose:runtime-isolation:live`	Live Vagrant/libvirt and WASI CLI instances
Released-product acceptance	0.2.4 downloaded-binary acceptance	Linux amd64, single-host supported profile

See Verification and examples for the complete test matrix and the distinction between model, plan, dry-run, source-adapted, and live production evidence.

Optional: run the live Contract probes



This path builds images, starts disposable Docker workloads, and downloads pinned images when absent. Use the presenter path when time, network, or host capacity is uncertain.

```
task testing:compose:contract-probes:live
```

The harness builds a standards-native GraphQL/gRPC producer and an independent consumer, then deploys the canonical Mosquitto, NATS, Kafka, and RabbitMQ DOTs. It proves GraphQL query,

mutation verification/cleanup, and GraphQL-over-SSE; all four finite Proto3 `NO_SIDE_EFFECTS` gRPC shapes through reflection; and correlated MQTT, NATS, Kafka, and AMQP 0-9-1 delivery. It records redacted evidence and destroys composition-owned resources on success, failure, or interruption.

Five-minute presenter script

1. Show the Contract-to-Anchor-to-wire diagram and explain the four ownership boundaries.
2. Run `dotx validate DOT-Repository` to establish that the catalogue is the checked source of truth.
3. Inspect Keycloak to demonstrate one reusable product identity with several deployment choices.
4. Print the 21-wire plan summary and highlight the four Contract standards and eight interaction forms.
5. Inspect the Kafka wire to distinguish Contract semantics from its resolved broker endpoint.
6. Render the APISIX/Keycloak/OAuth lifecycle with `--dry-run`.
7. Finish on the evidence table and state the supported boundary explicitly.

Keep the published release notes and verification matrix open as fallback evidence if a local network or runtime is unavailable.

Supported boundary

The stable profile supports Linux amd64 and private single-host composition state. Multi-host orchestration, automatic cross-platform gateway provisioning, general stateful-data migration, AMQPS, AMQP 1.0, native arm64 and non-Linux executables, the DOT-Registry REST API, and joint SFC or partner acceptance are separate activities. Unsupported combinations fail closed rather than being silently approximated.

All eleven MARE PoCs have planning or source-adapted evidence. That evidence is not presented as acceptance of every unmodified partner workload. Continue with Architecture for system boundaries or DOT Anchors and Composition for authoring and lifecycle details.

Docker/Kubernetes four-Contract demo

This `next` development walkthrough reuses three canonical catalogue definitions: the standards-native provider, consumer, and official Mosquitto broker. Named deployment variants select Kubernetes for provider/broker and Docker for consumer. No product descriptor is duplicated.

The composition is named `docker-kubernetes-four-contracts`. Its template lives at `DOT-Testing/composition/docker-kubernetes-four-contracts/scenario.yaml`: this is a synthetic test graph, unlike the reusable catalogue examples `keycloak-apisix-oauth` and `keycloak-prometheus-openapi-planning`.



Provider and consumer are explicitly test-only components, not MARE analytics or vendor products. Real HTTP, GraphQL.js, gRPC-js and Mosquitto exchanges demonstrate interoperability mechanics, not producer acceptance, production

> security, or automatic protocol conversion. Probe clients use the actual consumer's Docker network namespace; its application process is an inert deployment target. MQTT traffic is generated by the correlated delivery probe, not the GraphQL application. The MQTT runner uses the pinned official `mosquitto_rr` client in MQTT 3.1.1 mode, subscribing and publishing on the same Contract-selected channel. It waits for SUBACK before publishing, ignores retained messages, uses a clean session, and requires the exact uniquely correlated payload. This proves broker delivery, not an application-level request/response implementation.

What runs where

Wire	Authoritative Contract	Kubernetes endpoint	Docker-side assertion
`rest-alerts`	OpenAPI 3.1, `GET /alerts`	Provider, NodePort `31080`	Expected HTTP status
`graphql-health`	GraphQL SDL, `Query.health`	Provider, NodePort `31400`	`/data/health` equals `true`
`grpc-health`	Protobuf, side-effect-free `AlertService.Check`	Provider, NodePort `31551`	Reflected response has `serving: true`
`mqtt-alert-delivery`	AsyncAPI 3.0, MQTT-bound alert channel	Mosquitto, NodePort `31883`	Schema-valid, uniquely correlated QoS 1 delivery

Anchors select Contract operations. Wires select participants and materialize resolved endpoints into the consumer's declared environment. Each wire declares `EXTERNAL_GATEWAY`: the selected DOT declares the NodePort Service, and Minikube provides the reachable node address. Docker normally isolates its bridge networks: the harness explicitly attaches only its disposable consumer to the existing `minikube` Docker network. Cross-runtime probes use `--network container:<consumer>` so they exercise that actual consumer route, not a different runner's network. The attachment is external to `dotx` state, and consumer destruction removes it; the harness verifies its removal. No host listener, firewall change, tunnel, or Minikube-container modification is made.

Prepare before presenting

Use the full checkout in Contributor guide, and run commands from its root. Requirements: Linux amd64, Go and Task to install this checkout's CLI, Git, Bash, jq, curl, GNU coreutils, `flock` (util-linux), Docker with Compose, kubectl, and a running local Minikube cluster using its Docker driver and `minikube` Docker network. The Kubernetes context must be `minikube`. The deterministic demo checks also require ShellCheck and Python 3. On Ubuntu, install the `shellcheck` package before running `task testing:demo:checks`. The introductory Prometheus quickstart still needs no Go or Kubernetes.

```
git submodule update --init --recursive
task ci:prepare
git rev-parse HEAD
git -C DOT-CLI rev-parse HEAD
git -C DOT-Repository rev-parse HEAD
go version -m "$(command -v dotx)"
docker info --format '{{.ServerVersion}}'
docker compose version
kubectl config current-context
```

```
kubectl get nodes --request-timeout=10s
minikube ip
```

`dotx 0.2.4` alone is insufficient: this example uses post-release development changes. Install the pinned submodule and keep the checkout fixed. Evidence records revisions, CLI binary SHA-256, root/CLI/catalogue working-tree patch hashes, and provider/consumer image IDs. Non-empty patches mean the commit alone does not describe the tested checkout. Broker/probe images use digest references; the test images are locally built from pinned Dockerfiles/dependency locks, not claimed published immutable images. The provider uses `imagePullPolicy: Never` after explicit loading; the harness verifies actual running image IDs. Rehearse with network access for metadata/dependency/image downloads; caching does not promise fully offline execution.



Use an isolated demo machine/network. NodePorts are plaintext; MQTT permits anonymous access and disables persistence. Do not expose them to untrusted networks. These are demo defaults, not production authentication, TLS, retention, or availability settings. Never delete an unrelated namespace or Service.

Inspect without deploying

```
bash DOT-Testing/e2e/multi-runtime-demo/test.sh --dry-run
```

This resolves three DOTs/four wires, writes and verifies a lock, checks both platforms and all four standards, and renders deployment actions in a private directory under `DOT-Testing/e2e/target/multi-runtime-demo/`. The template uses documentation-only address `192.0.2.1`; never manually apply that template.

Run the working example

```
bash DOT-Testing/e2e/multi-runtime-demo/test.sh --live
```

Equivalent Task: `task testing:demo:docker-kubernetes-four-contracts:live`. The harness locks the real Minikube address, builds/loads images, creates an isolated namespace and state cache, deploys through `dotx compose apply --locked`, checks readiness, and probes each wire with a 45-second timeout. It verifies recorded status, destroys the composition, removes its namespace, and checks for owned Docker leftovers.

Expect `PASSED` lines for `rest-alerts`, `graphql-health`, `grpc-health`, and `mqtt-alert-delivery`, plus exit zero and `cleanup=PASSED`. Inspect the printed run directory's `plan.json`, `render.txt`, `status.txt`, and `evidence.json` to explain the locked plan, endpoint materialization and observations. Evidence JSON contains no passwords/tokens or response bodies. Each run retains its evidence; the top-level `evidence.json` is only the latest result, possibly an injected failure.

Rehearse the complete presentation

Present Prometheus first, authenticated Keycloak/APISIX/OAuth second, then the cross-runtime

standards example:

```
task testing:demo:dry-run
task testing:demo:mqtt-suback:live
task testing:demo:rehearse
```

Rehearsal repeats all three live paths twice. Prometheus covers default/overridden ports and failure cleanup. OAuth verifies HTTP 401 without a token, token acquisition, protected access, native Secret references, status, and cleanup. The separate MQTT regression uses the actual pinned client and a loopback-only fault-injection peer: delayed, denied, or absent SUBACK, stale retained messages, and incorrect correlated payloads must behave correctly. This peer is not a replacement for the full composition's real Mosquitto acceptance. Its ephemeral credentials and Keycloak development mode are not production defaults. The final mixed-runtime run deliberately fails after deployment: exit 42 and successful cleanup are required. Two further live runs receive actual SIGINT and SIGTERM after deployment and network attachment. They must exit 130/143, record cleanup success, and pass independent namespace/container/network checks. Run those checks separately with `task testing:demo:interruption-cleanup`.

The rehearsal prints one per-run `summary.json` under `DOT-Testing/e2e/target/rehearsal/run-*/`, with a separate log for every step, exit codes, durations, and before/after checkout and CLI fingerprints. Tracked patches and untracked-file content fingerprints are recorded for all three checkouts; no source or credential contents are copied into the receipt. Changed inputs during rehearsal fail acceptance. The convenience `latest.json` points to that per-run receipt, including failures; it is not a green badge. A dirty checkout remains explicitly labelled by its recorded status.

This is pre-presentation acceptance, not the short live talk itself. A per-user live-run lock rejects overlapping instances before deployment, including across checkouts using the same runtime/cache environment. `DOTX_DEMO_LOCK_DIR` is an override for isolated lock tests only; do not override it to run concurrent copies of this composition.

Recover safely

Normal failure, Ctrl-C and termination attempt cleanup. Do not use `kill -9` or delete state while cleanup runs. After interruption, use the exact printed path:

```
RUN_DIR=/absolute/path/printed/by/the/failed/run
XDG_CACHE_HOME="$RUN_DIR/cache" dotx compose status "$RUN_DIR/scenario.yaml"
XDG_CACHE_HOME="$RUN_DIR/cache" dotx compose destroy "$RUN_DIR/scenario.yaml"
```

Inspect the namespace in that run's `render.txt`; delete only that test-owned namespace if it remains. An apply failure may already have rolled back with no applied state; independently verify resource absence. Never prune Docker, remove Minikube, or delete unrelated volumes. Find the owner of an occupied NodePort before proceeding. Fix Docker user permissions before rerunning. Gateway timeouts require checking node IP, routing/firewall, readiness and Services; planning does not prove reachability. Contract download failures require repository access or verified bundle-local assets, never bypassing SHA-256 checks.

Catalogue readiness appendix

```
task testing:contracts:readiness
jq '.dots | group_by(.status) | map({status: .[0].status, count: length})' \
  DOT-Testing/e2e/target/catalogue-contract-readiness.json
```

Audit baseline: 69 valid definitions, 29 strict interface-profile passes and 40 gaps. Use the generated report for the actual checkout. Broker-only and negative fixtures do not necessarily need a business API. Official/source-derived Contracts, source-adapted PoCs, planning fixtures, stubs and producer doubles remain distinct evidence levels. See catalogue `CONTRACT-AUDIT.md` and Verification and examples. This demonstration does not promote the entire catalogue to production-ready.

Contributor guide

This page covers building DOT-Design and running its shared local/GitHub gates. To use an existing DOT without installing development toolchains, start with Getting started.

Prerequisites

Tool	Required for	Project baseline
Java and Maven	`DOT-Domain`, schema generation, and Java tests	Java 27 (`27.0.0+35-open` in `.sdkmanrc`); Maven 3.10.0 from `PATH`
Go	Building and testing `dotx`	Go 1.27.2
Task	Repository command aliases	Optional but recommended
Docker, Kubernetes tools, Helm, or Vagrant with libvirt/KVM	Only the matching deployment adapters and live scenarios	Install only what you intend to execute
Antora, the Antora PDF extension, Asciidoctor PDF, and Mermaid CLI	Building the HTML/PDF documentation site and Mermaid diagrams	CI uses Node.js 26 and Ruby 4.0; `task docs:check` reports the locally installed tool versions; see Build the documentation

Check out the repository

```
git clone https://github.com/MARE-EU-Project/DOT-Design.git
cd DOT-Design
git submodule update --init --recursive
```

The source repository and its two submodules may require access granted by the MARE-EU-Project organization. `DOT-CLI` and `DOT-Repository` remain separately cloneable; the generated Schema and high-level Skill classifications are tracked inside `DOT-Repository` and require no additional checkout.

Verify the domain model

The domain module uses plain JUnit tests and does not start a service:

```
mvn -pl DOT-Domain test
```

Run the complete Java build with:

```
mvn clean install
```

```
# equivalent Task alias
task mci
```

The build generates CycloneDX JSON and XML software bills of materials during the Maven `verify` phase.

Install and test dotx

`DOT-CLI` is an independent Go module and is not part of the Maven reactor:

Install the version-aligned standalone release with Go 1.27 or later:

```
go install github.com/MARE-EU-Project/DOT-CLI/cmd/dotx@v0.2.4
dotx version
```

The GitHub release also provides a checksummed Linux-amd64 binary, build metadata, and a manifest that pins the exact DOT-CLI source plus the DOT-Design and DOT-Repository acceptance revisions. For source development or the complete cross-repository suite, build the checked-out submodule instead:

```
task cli:build
task cli:test
task cli:install
dotx --version
```

`task cli:install` installs the pinned checkout into `GOBIN` or `GOPATH/bin` and installs Bash completion. Keep that directory on `PATH`; project tasks, test harnesses, examples, and CI all invoke the same `dotx` command rather than a repository-local binary.

Run GitHub workflow gates locally

GitHub Actions and local pre-push validation share the root Taskfile test bodies. Actions still provisions clean toolchains and publishes artifacts, but the build, validation, conformance, coverage, runtime, and documentation commands are not CI-only shell fragments.

```
task ci:prepare
task ci:deterministic
task ci:compatibility-matrix
```

`ci:prepare` installs the pinned DOT-CLI checkout and verifies that `dotx version` matches the root `VERSION`. `ci:deterministic` runs every deterministic non-live release workflow gate, including Java and Go tests, schema drift, official Contract validation, repository and Anchor conformance, Composition v1, lifecycle recovery, upgrade, all PoC dry-runs, external-DOT rendering, coverage, Antora, and PDF generation. Individual equivalents are available as `ci:build`, `ci:contract-conformance`, `ci:repository-conformance`, `ci:coverage`, and `ci:docs`.

On a Linux host with all runtime prerequisites, run the mutable workflows too:

```
task ci:live
task ci:all
```

The live gate requires Docker, Minikube, Kubernetes, Helm, Wasmtime, Vagrant/libvirt, writable KVM access, Rust 1.98.1 with the `wasm32-wasip2` target, and network access for the first official WASI HTTP sample build. The shared task fetches the exact pinned `sample-wasi-http-rust` revision into an ignored test cache, builds it with Cargo, and passes it to the runtime test, so local and GitHub executions cannot silently omit the WASI HTTP assertions. Set `WASI_HTTP_COMPONENT` only to reuse an already built component. The live gate performs runtime mutation and cleanup; `ci:deterministic` is the normal pre-push default.

For a Vagrant runtime on Ubuntu, install or reconcile the complete provider boundary and then open a new login session so the `libvirt` and `kvm` groups are active:

```
dotx tool install vagrant
dotx tool check vagrant
```

The installer selects `qemu-kvm` when the distribution provides it and falls back to `qemu-system-x86` on releases such as Ubuntu 26.04. The readiness check also verifies the pinned `vagrant-libvirt` plugin, the `qemu:///system` connection, the default libvirt network, and `/dev/kvm` access.

Validate a DOT

`dotx validate` reads `dot.json` at the project root, falling back to `dot/dot.json`. The selected file is the unique DOT deployment definition; unreferenced Dockerfiles, Compose files, Kubernetes manifests, Helm charts, and scripts are not inferred.

```
dotx validate DOT-Repository/DOTs/testing/contract-standards/provider \
  --schema DOT-Repository/Schema/schema.json
dotx validate DOT-Repository/DOTs/external/Keycloak-DOT \
  --schema DOT-Repository/Schema/schema.json --profile reusable
dotx inspect DOT-Repository/DOTs/testing/contract-standards/provider --json
```

Base validation checks the JSON Schema and DOT-level Anchor, Contract, service-port, CDI, and sensitive-default relationships. Every declared Contract must either be complete in `dot.json` or resolve to an asset below that DOT's `contracts/` directory. The validator never searches repository-level Contract directories or sibling DOTs. A bundle can therefore be copied and validated independently with an explicit schema path. A URL-backed definition declares one structured payload:

```
"payload": {
  "url": "https://raw.githubusercontent.com/vendor/project/v1.2.3/openapi.yaml",
  "sha256": "0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef"
```

```
}
```

Raw Contract text and the former flat `payloadUrl`/`payloadSha256` shape are rejected. Validation downloads the exact bytes, rejects non-HTTPS or mutable raw-GitHub branch URLs, verifies the digest, parses the declared standard/format, and resolves Anchor selectors. Missing IDs, incomplete declarations, digest mismatches, and unresolved selectors always fail validation. Use `--profile reusable` before publishing a stable reusable DOT. That profile additionally requires at least one pinned Contract, rejects unused Contracts, and requires every deployment to bind each Contract-backed provided Anchor exactly once through a service that declares the same Contract. Explicit contractless Anchors retain base-validation evidence only; they are never silently promoted to Contract-backed interoperability.

For an external product, prefer a complete root Contract declaration pointing to an official versioned vendor raw file. A full Git commit or release-version tag is accepted for `raw.githubusercontent.com`; `main`, `master`, and other mutable branch names are rejected. When no official raw machine-readable Contract exists, do not fabricate vendor provenance: retain a clearly attributed derived/adapter Contract until it can be migrated. Adjacent wrappers keep internal, testing, sample, and composition fixtures self-contained. Shared Contract UUIDs must resolve to identical metadata and payload bytes in every bundle; repository validation rejects conflicts. Use `dotx validate --offline` only when a matching local wrapper is present; offline validation verifies that its identity, metadata, and payload SHA-256 match the URL-backed declaration without contacting the vendor. For private raw GitHub URLs, `dotx` reads `MARE_READ_ONLY_TOKEN`, falling back to `GITHUB_TOKEN`, and sends it only to `raw.githubusercontent.com`; credentials must never appear in `payload.url`. Inspection provides a normalized, machine-readable view without deploying anything.

Validate and plan a composition

The canonical example connects Keycloak, APISIX, and an OAuth client:

```
dotx compose lock \
  DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml --check

dotx compose validate \
  DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml

dotx compose plan \
  DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml

dotx compose plan \
  DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml --json

dotx compose apply \
  DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml --dry-run
```

All five commands above are non-mutating. `lock --check` verifies the committed content-addressed input graph; `validate` answers whether references and compatibility can be resolved; `plan` shows the exact instances, endpoints, security status, and materializers; `apply --dry-run` renders

supported runtime actions without writing composition state.

Read Validation, planning, and lifecycle before a live apply.

Verify the authoritative repository

Recursively discover and validate every definition. External/internal paths record ownership; Contract-backed external/internal bundles receive the strict reusable profile:

```
dotx validate DOT-Repository
dotx repository inspect \
  d4a70000-0000-4000-8000-000000000001@1.3.0 \
  --repository DOT-Repository --json
```

The same command is the required DOT-Repository GitHub Actions gate for every change below `DOTs/` or `.gitmodules`. The filesystem is authoritative; no separate inventory or validation wrapper is maintained.

Run `task testing:contracts:portability` to copy every bundle into an isolated temporary directory and validate it offline against the explicit catalogue schema. This is also part of `ci:repository-conformance`; it prevents a valid repository checkout from concealing a DOT's missing local Contract asset.

Run the composition conformance gate

```
task testing:conformance
```

This gate combines Java domain tests, Go tests, repository-wide schema validation, Contract checks, composition validation/planning, and dry-run scenario checks. It does not claim that external partner systems or production networks were exercised.

Run only the versioned composition production profile with:

```
task testing:compose:acceptance
```

This gate writes revision-pinned, secret-free JSON evidence below `DOT-Testing/e2e/target/`.

Build the documentation

Install the documentation toolchain once:

```
task docs:setup
```

Then generate the Antora HTML site and PDF export:

```
task docs:check
task docs:build
```

Generated artifacts are written under `DOT-Docs/build/site` and are intentionally ignored by Git. To build and serve the site locally:

```
task docs:run
```

Open `http://localhost:8000`. Do not edit files under `DOT-Docs/build`; update the AsciiDoc sources instead.

Mermaid sources are stored as Antora attachments and rendered locally by `task docs:diagrams`. The renderer produces SVG for the HTML site and high-resolution PNG for PDF export; both outputs are committed so a clean checkout can resolve every Antora resource. Each source includes an accessible title and description, and each AsciiDoc image macro supplies alternative text.

`task docs:build` regenerates the diagrams before building either output. The toolchain is version-pinned and does not send architecture content to an external rendering service. Edit the `.mmd` source, never the generated SVG or PNG, then run the full documentation build to verify both renderers.

Where to continue

- Guided demo for a presentation-ready walkthrough from catalogue validation to cross-standard planning and optional live probes.
- Architecture for system and ownership boundaries.
- DOT Anchors and Composition for the model overview and task-oriented composition guides.
- Verification and examples for evidence levels and MARE PoC scenarios.
- DOT-Domain for Java model details.
- Integration handoff for catalogue and composer implementers.

Architecture

DOT-Design separates reusable component description from deployment-instance topology. That separation allows a DOT to be authored and validated once, then instantiated in multiple compositions without embedding addresses, credentials, or orchestrator-specific connection objects in the DOT aggregate.

The diagrams use a consistent visual language: blue identifies authored or persistent artifacts, teal identifies validation and resolution activities, amber identifies decisions or incomplete evidence, green identifies successful outcomes, and red identifies rejection. Labels and line styles carry the same meaning when the documentation is printed without color; dashed lines indicate optional, indirect, or evidence-only relationships.

System context

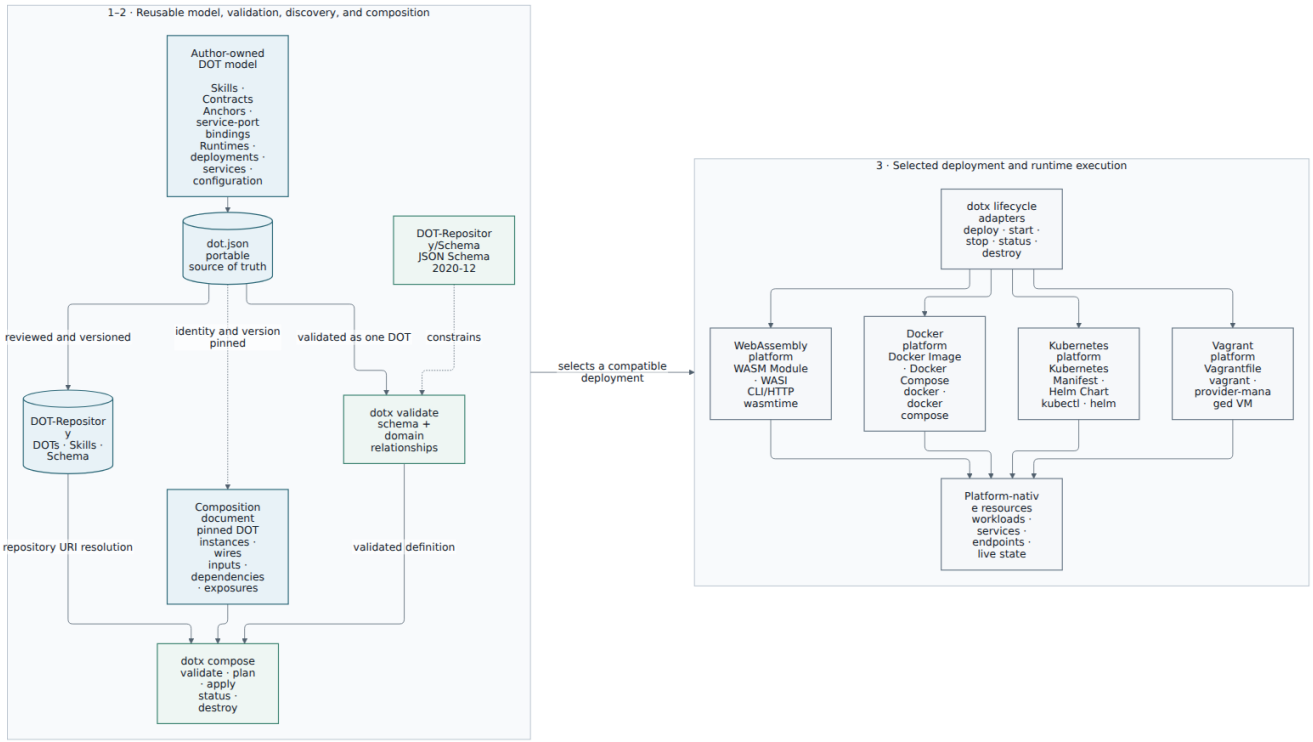


Figure 2. DOT-Design system context and runtime support

`DOT-Repository` is the authoritative catalogue of reusable definitions. `dotx` is the reference validation, resolution, deployment, and composition client. Platform-native systems remain responsible for workload execution and live state.

The runtime portion deliberately distinguishes **where** a workload runs from **how** it is deployed. The domain model supports four `PlatformEngine` values and six compatible manifest mappings:

- Docker accepts `DOCKER_IMAGE` and `DOCKER_COMPOSE`;
- Kubernetes accepts `KUBERNETES_MANIFEST` and `HELM_CHART`;
- Vagrant accepts `VAGRANTFILE`; and
- WebAssembly accepts `WASM_MODULE`.

The current `dotx` lifecycle implements Docker Image, Docker Compose, Kubernetes manifest, Helm chart, Vagrantfile, and WebAssembly execution. The WebAssembly adapter uses Wasmtime with an explicit `WASI_CLI` or `WASI_HTTP` world, verifies the module's SHA-256 before execution, and grants only declared guest environment and directory capabilities.

Source-of-truth boundaries

Concern	Authoritative source	Explicitly excluded
DOT identity and supported capability	`dot.json`: `id`, `name`, `version`, Skills, and Contracts	Composition-local instance names and live state
Contract semantics	OpenAPI, AsyncAPI, Protobuf, or GraphQL Contract payload	Repeated protocol, media, schema, or security fields on a contract-backed Anchor
Logical composition point	`Dot.anchors[]`	Hosts, URLs, credentials, placement, health, and policy decisions
Deployable endpoint declaration	`DeploymentService.ports[]` and `anchorBindings[]`	Allocated addresses, forwarded ports, and certificates
Runtime configuration surface	`DeploymentService.config`	Operator-supplied values already applied to one deployment
Multi-DOT topology and readiness	Composition document wires and `dependsOn`	Changes to either participating `dot.json`
Runtime state	Platform-native resources plus private `dotx` composition state	Contract or Anchor metadata

Repository architecture

```
DOT-Design
 ├──── DOT-Domain       Java model and validation
 ├──── DOT-CLI          independent Go CLI and reference executor
 ├──── DOT-Repository   DOTs, high-level Skills, and generated Schema (submodule)
 ├──── DOT-Testing      conformance, composition, interoperability, and E2E evidence
 └──── DOT-Docs         Antora sources, UI, playbooks, and generated diagrams
```

The Maven build contains the `DOT-Domain` Java module. `DOT-CLI` has its own Go module and lifecycle. `DOT-Testing` is deliberately outside both module hierarchies. `DOT-Repository` is the only admitted source for valid deployable DOT definitions used by shared tests. It also owns the generated JSON Schema and the optional high-level Skill classification catalogue, avoiding separate repositories for assets that evolve with the DOT catalogue.

DOT aggregate

`Dot` is the aggregate root. Its primary relationships are:

```
Dot
 ├──── Skills[]                human-readable capabilities
 ├──── Contracts[]             authoritative interaction definitions
 ├──── Anchors[]               stable logical composition points
 └──── Runtimes[]
     ├──── Platform            where execution occurs
     └──── Deployments[]
         ├──── DeploymentManifest   how deployment occurs
         └──── DeploymentService[]
             ├──── Contract references
             ├──── ServicePort[]
             ├──── AnchorBinding[]
             └──── DeploymentConfig
```

`Skill` and `Contract` are independently identified domain entities. Root `Dot.contracts` carries authoritative Contract declarations (with legacy UUID-only catalogue references accepted during migration), while deployment services continue to reference those Contracts by ID only. Anchors, Runtimes, Deployments, services, bindings, and configuration declarations are embedded values owned by the DOT aggregate.

The class-level `@ValidAnchor` constraint delegates aggregate traversal to `AnchorValidator`. It enforces Anchor identity, Contract membership, selector syntax, and service-port binding consistency without placing cross-reference logic in `Dot`.

Contract, Anchor, binding, and wire

These four concepts form a resolution chain but have different owners:

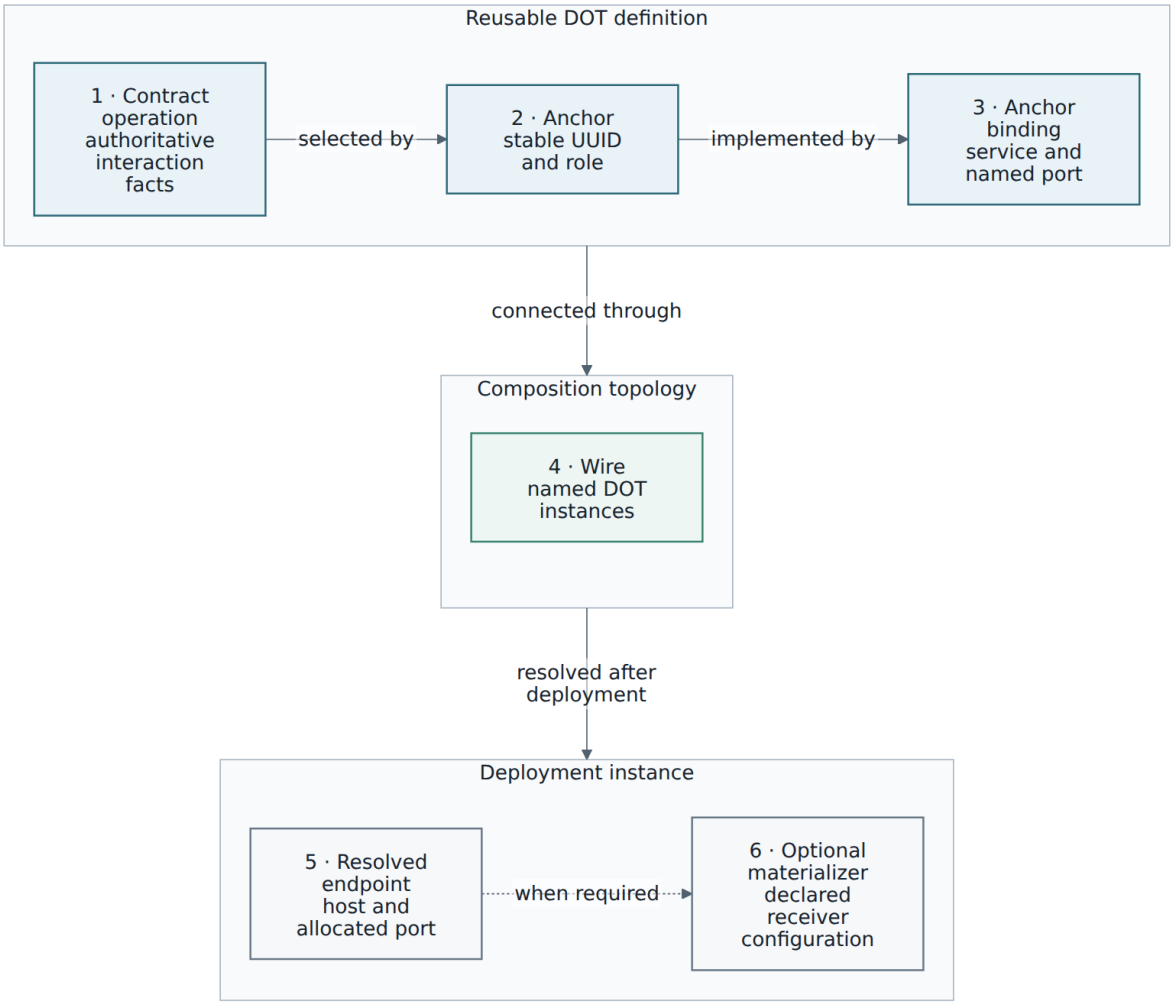


Figure 3. From an authoritative Contract operation to deployed configuration

- The **Contract** defines the interaction.
- The **Anchor** identifies one logical participation point.
- The **binding** says which declared service port implements that Anchor.
- The **wire** connects named deployment instances.
- The optional **materializer** writes a resolved value into configuration that the receiving service has already declared.

See DOT Anchors and Composition for the field model, compatibility rules, and examples.

Runtime and deployment separation

The model separates two orthogonal questions:

Concept	Field	Answers
`PlatformEngine`	`Runtime.platform.engine`	Where does the workload run? For example Docker, Kubernetes, Vagrant, or WebAssembly.

Concept	Field	Answers
`DeploymentManifestType`	`Deployment.manifest.type`	How is it deployed? For example Docker image, Kubernetes manifest, Helm chart, Vagrantfile, or WASM module.

Every manifest type declares its target platform. Runtime validation rejects a deployment whose manifest mechanism does not align with its platform.

`DeploymentService` is the unit that binds a deployed service to Contracts, named ports, Anchors, and supported configuration. Configuration is service-scoped; there is no deployment-level shared configuration surface.

Contract resolution

The reference resolver maps an Anchor selector to an authoritative Contract element:

```
AnchorContract(id, selector)
        |
        v
load Contract by UUID
        |
        v
standard-specific resolver
        |
        v
identifier + interaction + protocols + media types
          + input/output schema references + security scheme names
```

Resolution is read-only. Derived facts are returned to consumers and never written back into the Anchor. Contractless Anchors already carry their explicit direction, protocols, and media types and therefore have no selected Contract element.

Composition resolution

The stable Composition v1 executor uses a compile-like pipeline:

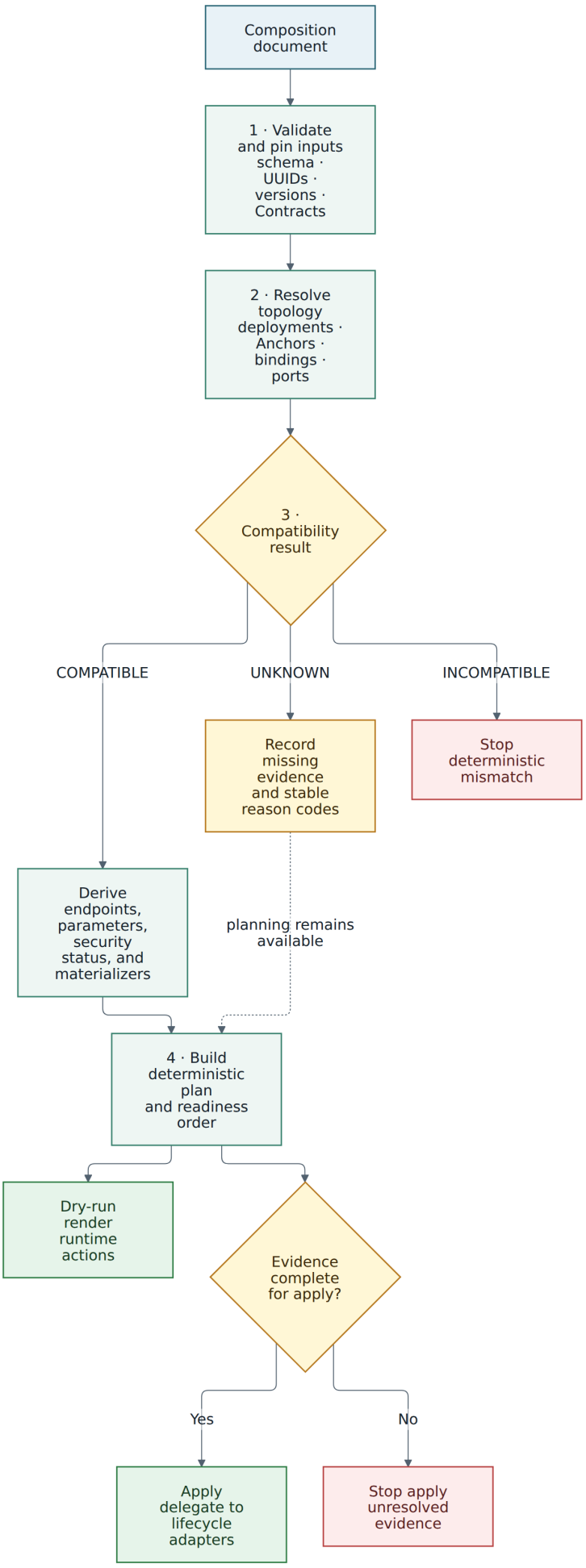


Figure 4. Fail-closed composition resolution pipeline

1. Validate the strict Draft 2020-12 composition shape.
2. Load each local DOT source and verify the pinned UUID and version.
3. Load each Contract input and verify its UUID.
4. Select one deployment for every named DOT instance.
5. Resolve Anchor references and service-port bindings.
6. Evaluate three-state compatibility: `COMPATIBLE`, `INCOMPATIBLE`, or `UNKNOWN`.
7. Derive connection endpoints, explicit cross-platform network ownership, parameter values, security status, and materializers.
8. Build a deterministic plan and readiness order.
9. On apply, prepare configuration and delegate workload operations to the existing single-DOT lifecycle adapters.

Planning may retain `UNKNOWN` with reason codes so missing evidence is visible. Apply rejects `UNKNOWN` rather than treating absence of evidence as compatibility.

Security boundaries

DOT-Design models declarations and explicit grants; it does not infer trust.

- Contract security requirements remain in the Contract.
- Sensitive deployment values are supplied by reference, not embedded in a composition or persisted as resolved values.
- OCI capabilities, host namespaces, volumes, and CDI devices require exact DOT declarations and exact operator grants.
- A configured wire does not prove credential validity, authorization, remote attestation, policy compliance, or network reachability.
- An `EXTERNAL_GATEWAY` binding records operator ownership and `NONE` cleanup; it does not assert that `dotx` provisioned or verified the gateway.
- Cross-domain admission decisions belong to deployment or policy systems.

Active traffic and trust evidence must be collected by scenario-specific probes after deployment.

Catalogue and API boundaries

`DOT-Repository` versions reusable definitions and its catalogue metadata in Git. `dotx repository` validates and resolves those entries; `dotx compose` provides the executable compatibility reference. A future remote catalogue or API is free to use another language, framework, storage engine, and transport provided it preserves the domain semantics and passes the consumer acceptance profile.

`dotx compose` stores private, local lifecycle state under the user's cache directory. Per-composition locking prevents concurrent local lifecycle commands from racing that state. The current model is therefore explicitly single-host; shared orchestration state remains future work.

Design principles

- Reuse established Contract standards instead of inventing parallel protocol models.
- Keep stable logical identity separate from runtime location.
- Make adapters, transformations, privileges, and secret access explicit.
- Fail closed on ambiguity and missing security evidence.
- Preserve independently versioned DOT definitions; keep topology in composition.
- Treat generated plans and test evidence according to what they actually prove.

DOT Anchors and Composition

This guide explains how independently authored Dynamic Open Trust (DOT) components participate in composition by referring to authoritative Contracts rather than duplicating contract-defined facts. It distinguishes the stable DOT Anchor model from the stable, versioned `dotx compose` format and follows the complete path from a logical capability to a deployed connection.

For a technology-neutral consumer boundary, see integration handoff.

Read this first

Keep these three boundaries intact:

1. A **Contract** describes what can be exchanged.
2. An **Anchor** gives one Contract operation a stable DOT-local identity and role.
3. A **wire** connects deployed DOT instances and remains outside both DOTs.



A structurally valid, deployable DOT is not automatically interoperability-ready. `dotx validate <bundle> --offline --profile interoperable` additionally requires a Contract for every Anchor and a Contract-backed provided Anchor for each exposed service port. The older `reusable` profile preserves mixed evidence during migration. Neither profile discovers undeclared implementation endpoints or replaces live producer acceptance. The catalogue Contract audit records official sources, source-derived internal descriptions, licensing boundaries, and remaining gaps. From DOT-Design, `task testing:contracts:readiness` produces a per-bundle strict-profile report; known gaps remain failures, not an interoperability certification.

If a value changes between installations—for example a host, URL, port, credential, realm, broker, namespace, or device—it does not belong in an Anchor.

If you want to…	Start with
Publish a composable DOT	Authoring two composable DOTs, then Contract-backed Anchors and deployment binding.
Connect existing DOTs	Composition documents and DOT instances, direct request/response wires, and materializers.
Model messaging through a broker	Brokered wires and path and channel parameters.
Handle authentication or secrets	Security and secrets.
Operate a composition	Validation, planning, and lifecycle and troubleshooting.
Design a new 6G integration type	Anchor boundaries, extension points, and 6G security composition patterns.

Release status and supported profile

The DOT-owned Anchor model, contract selectors, service-port bindings, validation, generated schema, repository resolution, and conformance fixtures are release candidates. The separate composition document and its local lifecycle executor implement the stable `dot.mare6g.eu/v1` profile.

The current executable profile supports:

- local DOT sources pinned by UUID and version, and local Contract payloads pinned by UUID, in the composition;
- OpenAPI, AsyncAPI, Protobuf, and GraphQL contract selectors;
- contractless protocol/media declarations when no supported contract exists;
- named deployment selection and service-port bindings;
- explicit environment-variable and config-file materializers;
- explicit externally owned gateway bindings for cross-platform wires, with ownership and no-cleanup semantics recorded in plan, lock, and state;
- Docker Image composition networking and per-instance identity;
- Kubernetes and Helm deployment/readiness through the existing single-DOT lifecycle;
- reference-only process-environment and Kubernetes Secret inputs;
- rollback, private local state, resumable partial apply, status, and reverse-order destroy;
- contract-authorized active probes for safe OpenAPI operations, GraphQL query/mutation/SSE lifecycles, finite side-effect-free unary/client/server/ bidirectional gRPC calls, and correlated AsyncAPI MQTT/NATS/Kafka/AMQP delivery within one Docker network or Kubernetes namespace; and
- fail-closed handling of ambiguity, unknown compatibility, unsupported privilege grants, and unreachable cross-platform materialization.

The authoritative machine-readable boundary is `DOT-Testing/acceptance/composition/v1/profile.json`. Run `task testing:compose:acceptance` to compare the installed CLI help and emitted schema with that profile, verify every committed lock, exercise all twenty-one cross-standard wires and six runtime dry-run adapters, and emit revision-pinned, secret-free JSON evidence.

It is not a general multi-host production orchestrator. Shared state, composition-owned cross-platform gateways or tunnels, external secret-store plugins, multi-scheme authentication, unsafe or unbounded gRPC, AsyncAPI event protocols other than MQTT, NATS, Kafka, and AMQP 0-9-1, AMQPS/AMQP 1.0, and cross-platform probe adapters remain outside this profile. Safe anonymous OpenAPI probes, single-scheme OAuth 2.0 client-credentials probes, GraphQL query/mutation/SSE probes, all four finite side-effect-free gRPC shapes, and AsyncAPI MQTT/NATS/Kafka/AMQP delivery probes within a shared Docker network or Kubernetes namespace are supported.

Single-DOT live lifecycle evidence for a runtime adapter is not automatically multi-DOT composition acceptance. The profile records validation, planning, dry-run, single-DOT live lifecycle,

and composition live lifecycle separately for each adapter.

The five distinct objects

Object	Owner	Responsibility
Contract	DOT author	Authoritative operation, message, payload, protocol, and security description in OpenAPI, AsyncAPI, Protobuf, or GraphQL.
Anchor	DOT author	Stable UUID and composition role selecting one contract element, or declaring explicit communication facts when no contract exists.
Anchor binding	DOT author	Maps one Anchor UUID to a named port of a deployment service. It does not contain a live address.
Wire	Composer/operator	Connects one named DOT instance and Anchor UUID to another. It may select a provider, consumer, or broker service-port endpoint.
Materializer	Composer/operator	Explicitly writes a resolved connection value into configuration declared by a participating service.

Where each fact lives

Fact	Source of truth	Referenced by
API operation, message, schemas, protocol, media type, security requirement	Contract payload	`Anchor.contract.selector`
Logical composition point and provider/consumer role	`Dot.anchors[]`	Wire `from` or `to`
Service and named port implementing an Anchor	`DeploymentService.anchorBindings[]`	Resolved plan endpoint
Configuration the service permits a deployer to supply	`DeploymentService.config`	Deployment input or materializer target
Selected DOT version, topology, readiness, parameter values, and secret references	Composition document	`dotx compose` plan and lifecycle
Concrete runtime address and port	Explicit composition network binding or resolved deployment plan	Materializer or exposed-Anchor forwarding

Fact	Source of truth	Referenced by
Live reachability, authentication result, and policy decision	Runtime probe or external policy system	Operational evidence; never persisted as an Anchor fact

The ownership chain is:

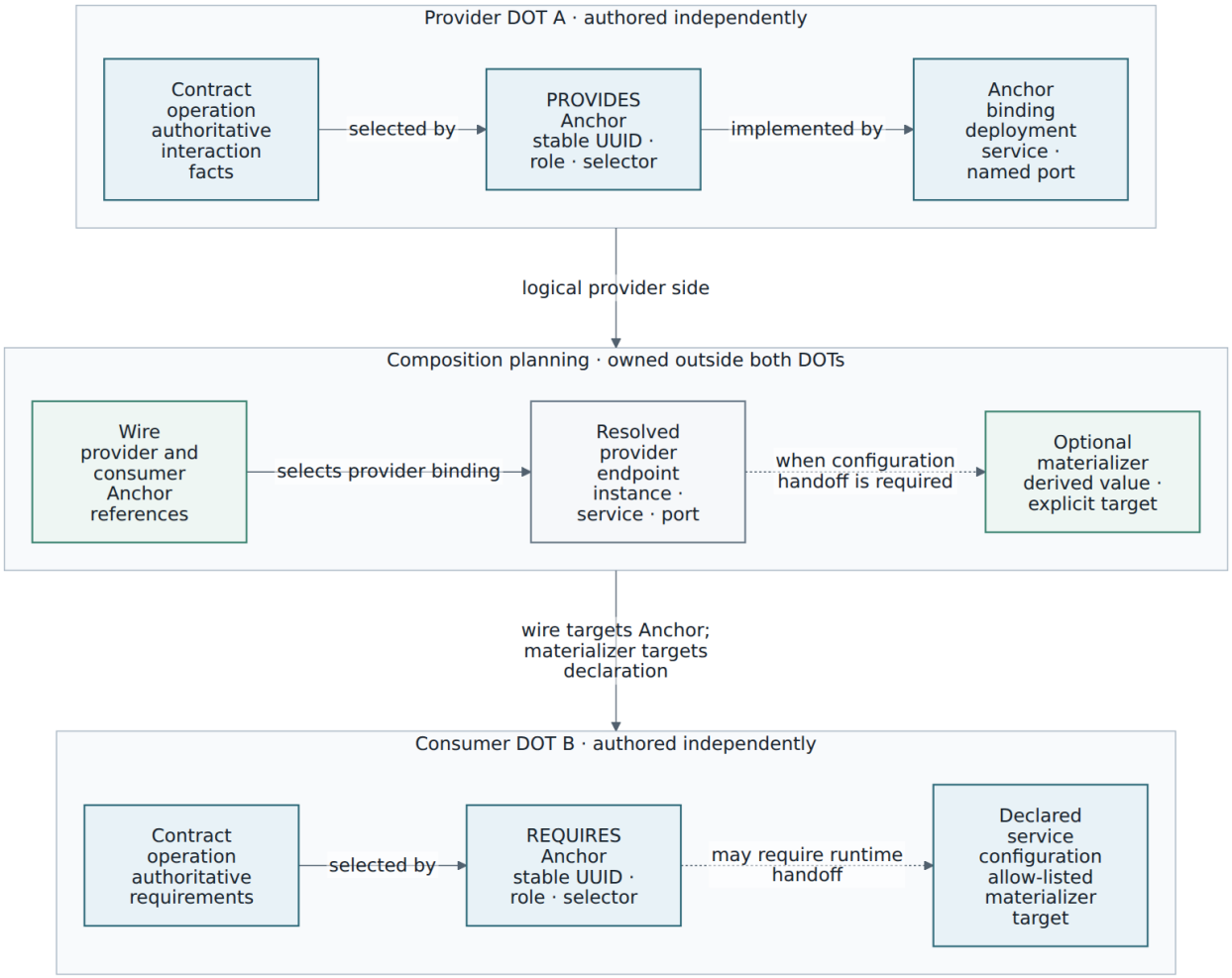


Figure 5. Provider-to-consumer ownership chain

These objects must not collapse into one schema. In particular, a wire is not part of either DOT, and a live URL is not part of an Anchor.

Resolution pipeline

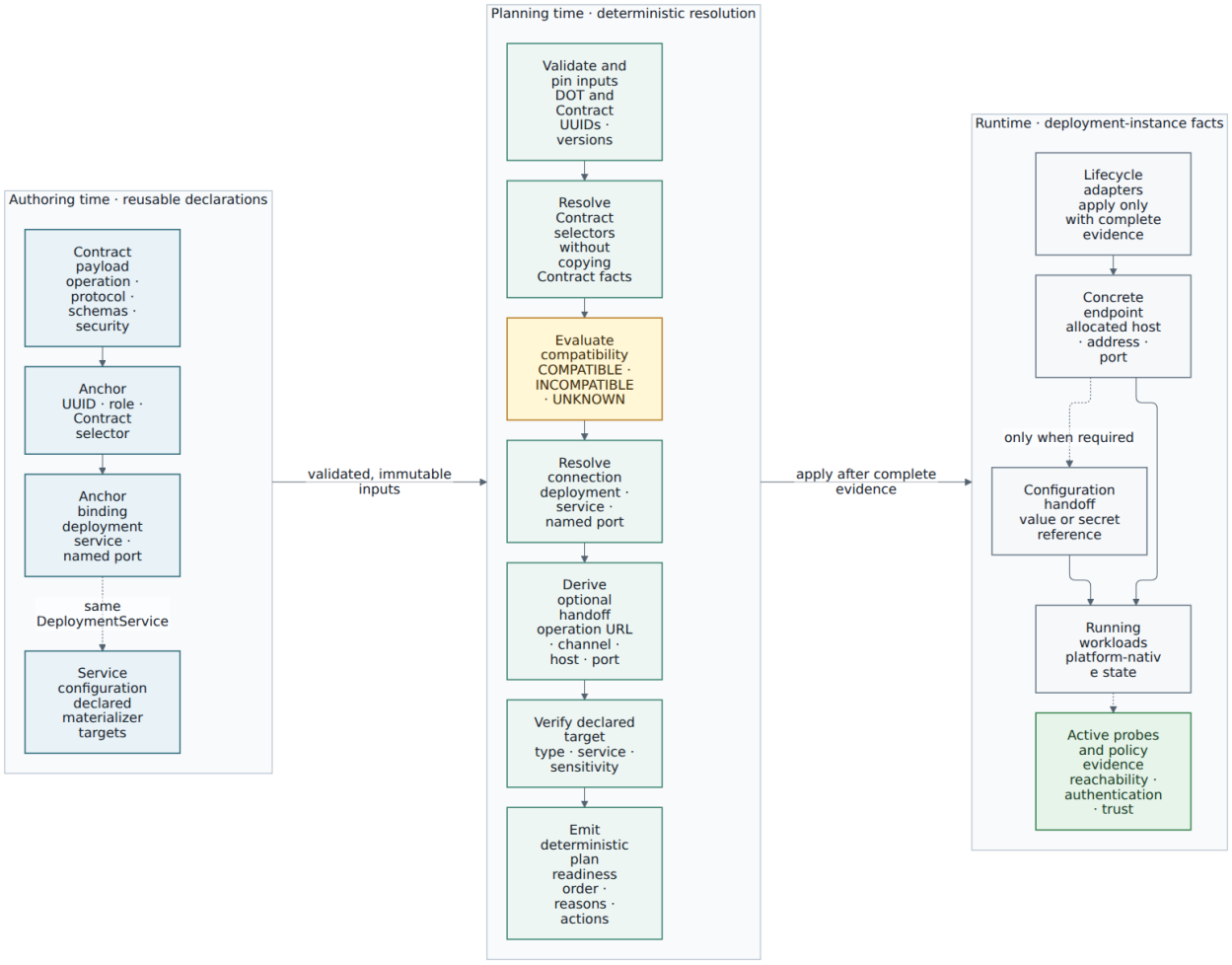


Figure 6. Authoring, planning, and runtime resolution

This resembles compilation: validate the independently authored inputs, link stable references, produce a deterministic deployment plan, and only then mutate runtime state. A failure at one phase is not evidence that a later phase was attempted.

Continue through the workflow

The detailed guide is organized around the work being performed:

Task	Guide
Publish stable composition points in a DOT	Authoring Anchors
Define pinned DOT instances and composition topology	Composition documents and instances
Connect endpoints, provide security inputs, and materialize configuration	Wiring, materialization, and security
Validate, plan, apply, inspect, and evolve a composition	Composition lifecycle and design guidance

Readers implementing a separate composer should also consult the integration handoff.

Authoring Anchors

This page covers the DOT publisher's side of composition. Start with the model overview if the distinction between Contracts, Anchors, bindings, and wires is not yet familiar.

Authoring two composable DOTs

Use this sequence when introducing a new connection:

1. Put the operation or message boundary in an OpenAPI, AsyncAPI, Protobuf, or GraphQL Contract. Use the contractless form only when none of those standards can describe the boundary.
2. Add a stable `PROVIDES` Anchor to the provider DOT and a stable `REQUIRES` Anchor to the consumer DOT. Select the authoritative Contract element from each Anchor; do not copy its protocol or payload fields.
3. On every deployable implementation, bind each Anchor UUID to the named `ServicePort` that implements it.
4. Declare every consumer configuration input on its `DeploymentService`, including sensitivity. This declaration is the allow-list for composition.
5. Validate each DOT independently with `dotx validate` before writing a composition.
6. Add named, UUID/version-pinned DOT instances and UUID-pinned Contract inputs to a separate composition document.
7. Connect the Anchor UUIDs with a wire. Add an explicit broker endpoint, parameter values, security binding, or materializer only when the connection requires it.
8. Run `dotx compose lock` and commit the generated lock beside the composition. Review digest changes as changes to the resolved DOT-to-DOT graph.
9. Run `dotx compose validate` to check the document and `dotx compose plan` to inspect the resolved topology. Treat `UNKNOWN` compatibility as missing evidence, not as success.
10. Use `apply --dry-run` before execution. After apply, use `compose probe` for safe Contract-derived traffic and `status` for native runtime status, configuration evidence, and the latest redacted probe evidence. Dedicated acceptance tests remain necessary for authentication, authorization, mutable operations, and application-specific outcomes.

The reusable three-DOT worked example is under `DOT-Repository/Compositions/keycloak-apisix-oauth/`: `composition.yaml` owns the wiring and references canonical DOT definitions and their local Contracts; `runtime/` holds non-secret instance inputs. The executable acceptance harness remains in `DOT-Testing/e2e/api-gateway-oidc/`.



Concrete identifiers in this guide use the canonical lowercase UUID text form with UUIDv4 version and IETF variant bits defined by RFC 9562. Deterministic values are retained where they identify executable repository fixtures, making the documentation traceable to the tested examples. Generate new UUIDs for production DOTs; do not reuse documentation or fixture identifiers.

Contract-backed Anchors

A contract-backed Anchor carries stable identity and selects one authoritative contract element. It deliberately omits protocol, direction, media type, payload schema, and security fields that the Contract already defines.

```
{
  "id": "a0000000-0000-4000-8000-000000000001",
  "name": "submit-alert-http",
  "role": "PROVIDES",
  "semanticTypes": ["urn:mare:security:alert"],
  "contract": {
    "id": "11111111-1111-4111-8111-111111111111",
    "selector": "#/paths/~1alerts/post"
  }
}
```

The opposite DOT declares its own `REQUIRES` Anchor. It may select the same Contract UUID or an independently authored Contract. Compatibility is deterministic when authoritative facts are comparable; otherwise the result is `UNKNOWN`, never an invented match.

Reading compatibility results

The plan compares the provider and consumer across independently explainable dimensions:

Dimension	Meaning
Role and direction	The connection is from `PROVIDES` to `REQUIRES`, and the selected interaction directions can communicate.
Semantics	Declared semantic-type URIs intersect when both sides provide them.
Protocol and media	Resolved protocol and media-type sets have a usable intersection.
Interaction	Request/response, publish/subscribe, streaming, query, mutation, or subscription behavior agrees.
Schema	Provider output and consumer input schema identities are compared in the correct direction when both select the same authoritative Contract; cross-Contract structural comparison remains `UNKNOWN`.
Security	Contract requirements are understood and, for execution, a complete supported binding has all required declared inputs.

`INCOMPATIBLE` means comparable evidence conflicts. `UNKNOWN` means evidence is missing or the current resolver cannot prove the dimension. Validate and plan may explain `UNKNOWN`; apply rejects it. An explicit mediator DOT is preferable to weakening compatibility when protocols or payloads genuinely differ.

Selector syntax

Standard	Selector	Example
OpenAPI	RFC 6901 URI-fragment JSON Pointer to an Operation Object	`#/paths/~1alerts/post`
AsyncAPI	RFC 6901 URI-fragment JSON Pointer to an Operation Object	`#/operations/publishAlert`
Protobuf	Fully qualified service method	`mare.security.AlertService.PublishAlert`
GraphQL	Root-field schema coordinate	`Mutation.publishAlert`

For OpenAPI and AsyncAPI, `/` in a JSON Pointer token becomes `~1` and `~` becomes `~0`. Concrete path or channel template values belong to a composition wire's `parameters`, not to the Anchor.

Supported Contract representation matrix

Standard	Contract `format`	Authoritative interaction	Composition evidence
OpenAPI	`YAML` or `JSON`	Request/response operation, including path and security requirements	One shared selector; `REQUEST_RESPONSE` for both participants
AsyncAPI	`YAML` or `JSON`	Message operation and Channel Object	Distinct complementary selectors; `SEND` and `RECEIVE`, plus the resolved channel address
Protocol Buffers (Proto3; including gRPC service declarations)	`PROTOBUF`	Fully qualified RPC and unary or streaming shape	One shared method selector; `REQUEST_RESPONSE`, `CLIENT_STREAM`, `SERVER_STREAM`, or `BIDIRECTIONAL_STREAM`
GraphQL	`GRAPHQL`	SDL root field	One shared schema coordinate; `QUERY`, `MUTATION`, or `SUBSCRIPTION`

`dotx compose plan --json` exposes the resolved `contractStandard`, `contractFormat`, `providerInteraction`, and `consumerInteraction` on each contract-backed wire. These fields are derived from the supplied Contract and therefore do not duplicate authoring metadata in either Anchor or composition. The deterministic `DOT-Testing/composition/contract-standards/` matrix connects one provider DOT to one consumer DOT through every row above.

Semantic types

`semanticTypes` is a list of URI identifiers describing the security or domain meaning of the interaction independently of its transport. Prefer stable, governed ontology or vocabulary URIs when one exists; a project URN is suitable for concepts that have no external vocabulary yet.

Current matching is deliberately conservative: both Anchors must declare at least one semantic type and their sets must intersect exactly. Missing values produce `UNKNOWN`; different values produce `INCOMPATIBLE`. The evaluator does not infer subclass, equivalence, or broader/narrower ontology

relations. Such reasoning belongs in a separately reviewed semantic-policy layer.

Contractless Anchors

Use a contractless Anchor only when no supported authoritative contract exists, for example a raw packet stream or a legacy metrics boundary. It must state the facts that cannot otherwise be derived:

```
{
  "id": "a0000000-0000-4000-8000-000000000006",
  "name": "packet-stream",
  "role": "PROVIDES",
  "semanticTypes": ["urn:mare:security:packet-stream"],
  "direction": "EGRESS",
  "protocols": ["TCP"],
  "mediaTypes": ["application/octet-stream"]
}
```

Contract-backed and contractless forms are mutually exclusive. A contract-backed Anchor must not repeat `direction`, `protocols`, or `mediaTypes`. A contractless Anchor must declare all three.

Role is not traffic direction

`role` answers who offers the selected capability:

- `PROVIDES` means this DOT makes the capability available;
- `REQUIRES` means this DOT depends on that capability.

It does not mean that bytes move in only one direction. An OpenAPI request/response operation has a providing server and requiring client while both send data. An AsyncAPI provider may select a `send` operation and its consumer a complementary `receive` operation. For a contract-backed Anchor, the resolver derives that interaction from the selected operation. Only a contractless Anchor states `INGRESS`, `EGRESS`, or `BIDIRECTIONAL` explicitly.

When two DOTs implement the same published Contract, referencing the same Contract UUID gives the strongest deterministic evidence. Independently authored Contracts are allowed, but identical-looking schema names do not prove structural compatibility across Contract identities; that comparison remains `UNKNOWN` until a schema-aware adapter or policy evaluates it.

Stable identity and version evolution

- The UUID, not `name`, identifies an Anchor in a wire.
- A rename or documentation correction preserves the UUID.
- A compatible DOT version preserves the UUID for the same logical Anchor.
- An incompatible semantic replacement receives a new UUID.
- Removal and any future `deprecated`/`replacedBy` metadata are publishing governance decisions; they are not inferred from contract changes.

- Names remain useful in logs and displays but are never foreign keys.

Binding an Anchor to a deployment

An Anchor is logical until a selected deployment maps it to a named service port:

```
{
  "id": "security-function",
  "name": "Security Function",
  "host": "security-function",
  "ports": [
    {"name": "http", "protocol": "HTTP", "port": 8080}
  ],
  "anchorBindings": [
    {
      "anchorId": "a0000000-0000-4000-8000-000000000001",
      "port": "http"
    }
  ],
  "config": {
    "environment": [],
    "configFiles": [],
    "volumes": []
  }
}
```

This says which logical service and named port implement the Anchor. The deployer still decides the concrete container DNS name, cluster Service, allocated address, forwarded port, certificate, or network path.

`@ValidAnchor` and `AnchorValidator` reject duplicate Anchor UUIDs, undeclared Contract references, invalid selector syntax, undeclared ports, undeclared Anchor bindings, and duplicate service-local bindings.

Runtime configuration portability

Every `DeploymentService` declares its supported configuration surface through `environment`, `configFiles`, and logical `volumes`. A composition may supply only values whose names or target paths the selected service declares. Required environment variables and files without defaults fail before runtime mutation; sensitive environment values require a `secretRef` instead of a literal. When an input is omitted, `dotx` reuses the non-sensitive `defaultValue` or `defaultContent` declared by that service. Explicit composition values and wire materializers override defaults; conflicting explicit inputs fail closed. Declarations marked `sensitive` are forbidden from embedding either kind of default and must use a deployment-time secret reference or runtime-native secret facility. Dry runs print resolved environment values in stable name order for every adapter, but replace every value declared sensitive with `<redacted>`.

Each composition input carries the stable `service` id it targets. Authors may omit `service` only when exactly one service in the selected deployment declares that environment name, file path, or

volume name; planning resolves and records the service explicitly. If two services declare the same item, omission is an error. Duplicate detection is service-local, so two services can intentionally receive different values for the same environment-variable name.

Environment variables and configuration files are rendered by every currently executable `dotx` deployment adapter:

Manifest	Environment	Configuration files
`DOCKER_IMAGE`	Container environment	Read-only bind mounts
`DOCKER_COMPOSE`	Per-service override using process-environment staging	Generated per-service Compose override mounts
`KUBERNETES_MANIFEST`	Exact workload/container patch; existing Secret keys remain `secretKeyRef`	Service-qualified ConfigMap or Secret plus exact workload/mount patch
`HELM_CHART`	Exact post-install workload/container patch; existing Secret keys remain native	Service-qualified ConfigMap or Secret plus exact workload/mount patch
`VAGRANTFILE`	Vagrant process/provisioner environment	Files provisioned into the managed VM and exposed to the Vagrantfile
`WASM_MODULE`	Allow-listed WASI guest environment; sensitive values inherited by name	Staged configuration root plus explicit Wasmtime directory preopens

Configuration remains service-scoped. A Docker image or Vagrant deployment is one workload or VM, so several logical services may safely share its declared configuration surface. Docker Compose, Kubernetes, and Helm may materialize several independent workloads. Their composition therefore supplies `deploymentInputs.workloads`: a Compose binding names one native service; a Kubernetes/Helm binding supplies exact native labels and optionally a container. Dotx intersects those labels with its composition-owned instance label and requires exactly one workload. Missing, duplicate, cross-runtime, ambiguous workload, and ambiguous container bindings fail closed. This prevents environment values, secrets, or mounted files from reaching the wrong security function without adding platform-specific selectors to the reusable DOT model.

```
deploymentInputs:
  workloads:
    - service: api
      kubernetes:
        matchLabels:
          app.kubernetes.io/component: api
        container: api
```

Storage and privilege configuration is deliberately runtime-native. Docker image deployments

support composition-scoped named volumes, explicit OCI bind sources, exact capability/host-namespace grants, and CNCF CDI devices. Compose payloads own their volume graph; Kubernetes and Helm own claims, storage classes, and volume topology; Vagrantfiles own disks and synced folders. A composer must not translate a host bind path into a cluster volume or silently grant privileged mode.

`WASM_MODULE` uses a reviewed Wasmtime/WASI profile. Each module is SHA-256 pinned and loaded from a relative/file or HTTPS URI. `WASI_CLI` executes command modules/components; `WASI_HTTP` serves a component on one declared TCP/HTTP service port bound to loopback. Dotx exposes only declared environment values, staged configuration files, and explicitly bound host directories. It does not translate OCI capabilities, host namespaces, or CDI devices into broader WASI permissions. Sensitive values are never printed or persisted. Because the Wasmtime CLI cannot enforce read-only permissions on a directory preopen, that request fails closed unless the host filesystem already supplies the boundary. The deterministic `DOT-Testing/composition/runtime-configuration/` matrix exercises every executable adapter twice: once using only declared non-sensitive defaults and once using explicit composition overrides. It checks the materialized default configuration document, the resolved service-scoped plan, and the runtime-specific dry-run renderer. The five native cases reuse the external Prometheus DOT; WASI retains a separate conformance fixture. The real Prometheus acceptance runner verifies loaded configuration and environment through its official status APIs, PromQL results and invalid-query responses, stop/start, Git acquisition, and failure cleanup:

```
task testing:compose:runtime-configuration
task testing:compose:runtime-configuration:live RUNTIME=docker-image
```

Select `docker-compose`, `kubernetes-manifest`, `helm-chart`, or `vagrantfile` for the other native mechanisms. These are ephemeral single-server examples, not production monitoring configurations. The `DOT-Testing/composition/service-workloads/` matrix verifies isolation between two services across Compose, Kubernetes, and Helm dry-run adapters.

Discovering and resolving Anchors

The authoritative test and reference definitions are stored in `DOT-Repository` and addressed as `repository://<dot-id>@<version>`. Consumers can inspect one immutable definition or validate the complete repository:

```
dotx repository inspect <dot-id>@<version> --repository DOT-Repository --json
dotx validate DOT-Repository
```

`dotx compose validate` and `dotx compose plan --json` resolve Anchor UUIDs, service-port bindings, and selected Contract elements conservatively. The plan reports `COMPATIBLE`, `INCOMPATIBLE`, or `UNKNOWN`; missing evidence is never treated as a match. Contract payloads remain authoritative and are not copied into Anchors or composition wires.

This command boundary is the reference behavior, not a requirement to embed dotx. Another catalogue or composer may use any language, framework, storage, or API shape if it preserves the same identities, fail-closed resolution, and ownership boundaries. See integration handoff.

Next, define the concrete participants and topology in a composition document.

Composition Documents and DOT Instances

A composition document is owned by the composer or operator, not by any member DOT. It selects immutable DOT identities and creates deployment instances whose topology may vary between environments.

Reusable composition catalogue

`DOT-Repository/Compositions/` distributes reusable multi-DOT graphs, reviewed locks, READMEs, and non-secret deployment inputs. Start with `keycloak-apisix-oauth/` (Keycloak, APISIX, OAuth client) or `keycloak-prometheus-openapi-planning/` (official Keycloak and Prometheus OpenAPI operations). Each directory has `composition.yaml` and `composition.lock.json`.

From the standalone DOT-Repository checkout root:

```
dotx validate . --offline
composition=Compositions/keycloak-apisix-oauth/composition.yaml
dotx compose plan "$composition" --locked --json
dotx compose apply "$composition" --locked --dry-run
```

Repository-wide composition validation requires the current source-built CLI; the released 0.2.4 binary predates this gate. It recursively checks declarations, semantic plans, adjacent locks, and READMEs without deploying or resolving Secrets. `task compositions:render` additionally renders every catalogue composition. There is no separate composition inventory or Scenario domain type.

A scenario is the broader use-case narrative and acceptance procedure described by the README. Synthetic matrices, negative controls, source-adapted PoC fixtures, executable harnesses, and evidence remain in DOT-Design's `DOT-Testing/`. See Getting started for the gateway's three Wires and Guided demo for live acceptance. Catalogue inclusion or a valid plan never implies production or live producer acceptance.

Composition documents and DOT instances

A composition is a separate versioned document. It names concrete instances of DOT definitions and pins their identities through the authoritative `DOT-Repository` catalogue. The `repository://<dot-id>@<version>` form is stable across checkout locations and prevents a scenario-local copy from drifting away from the reviewed definition:

```
apiVersion: dot.mare6g.eu/v1
kind: Composition
id: e3000000-0000-4000-8000-000000000001
name: api-gateway-oidc
version: 1.0.0
dots:
  - name: identity
```

```
    source: repository://d3000000-0000-4000-8000-000000000001@1.0.0
    dotId: d3000000-0000-4000-8000-000000000001
    dotVersion: 1.0.0
    deployment: kubernetes
  - name: gateway
    source: repository://d3a00000-0000-4000-8000-000000000002@1.0.0
    dotId: d3a00000-0000-4000-8000-000000000002
    dotVersion: 1.0.0
    deployment: kubernetes
    dependsOn: [identity]
contracts:
  - id: c3000000-0000-4000-8000-000000000001
    source: ./contracts/oidc-discovery.yaml
```

`name` identifies an instance inside this composition. `source` selects the repository definition by UUID and exact version. `dotx compose` locates the nearest `DOT-Repository` containing a `DOTs/` directory, discovers its definitions recursively, resolves the UUID/version to its reviewed bundle, and then verifies that `dotId` and `dotVersion` match the loaded `dot.json`. Neither a repository definition nor a local path can silently substitute a different DOT. `deployment` selects a deployment by its stable deployment ID or unambiguous manifest alias.

Local discovery is the convenience default. `--repository <path-or-Git-URL>` selects another authoritative catalogue and `--git-ref <revision>` selects a Git revision. Locked and runtime-mutating commands require a full 40-character commit SHA for remote repositories; branch and tag names are allowed only for unlocked, read-only exploration. Exact commits are cached in a private user cache by normalized repository identity and commit, so the same plan and lock can be reproduced offline. Plans, locks, and lifecycle state record only the credential-free identity and resolved commit. Credential-bearing URLs are rejected; authentication remains external through Git credential helpers, SSH agents, or environment-backed CI credentials.

Relative paths remain supported for authoring experiments and external Contract documents. Reusable stable, sample, and test-only DOT definitions belong in `DOT-Repository`, under their corresponding catalogue scope. Contracts remain authoritative protocol documents and are referenced independently; placing a DOT in the repository does not copy OpenAPI, AsyncAPI, Protobuf, or GraphQL facts into its Anchors.

`dependsOn` is a readiness graph, not a connection graph. A wire does not imply startup order. This distinction permits reciprocal peer wires and brokered topologies without creating false dependency cycles.

Composition field reference

The emitted Draft 2020-12 JSON Schema is normative for the serialized `v1` shape. This table is the study guide for its top-level ownership:

Field	Required	Meaning
`apiVersion`, `kind`	Yes	Fixed discriminators: `dot.mare6g.eu/v1` and `Composition`.

Field	Required	Meaning
`id`	Yes	Stable UUID of this composition, also used to derive local state and runtime identities.
`name`, `version`	Yes	Human-readable identity and version of the composition document.
`dots[]`	Yes, at least one	Named deployment instances. Every entry pins `source`, `dotId`, and `dotVersion`; published scenarios use an exact `repository://<uuid>@<version>` source. `deployment`, `dependsOn`, and `deploymentInputs` refine execution.
`contracts[]`	When a referenced Contract is needed	Maps an exact Contract UUID to a local Contract document. The document's embedded `id` must match.
`wires[]`	No	Connections between one `PROVIDES` and one `REQUIRES` Anchor. A composition with no wires can still expose a standalone capability.
`exposedAnchors[]`	No	Composition-local names for member Anchors that an operator may expose or forward.

Each wire requires `name`, `from`, and `to`. Both ends contain an instance name and Anchor UUID. Optional `endpoint` selects a third-party or otherwise non-default service port; `parameters` supplies only Contract-declared template values; `materializer` or `materializers` writes resolved values; and `security` binds Contract security requirements to declared participant configuration. Unknown fields are rejected.

Deployment inputs and native workload bindings

`deploymentInputs.environment`, `configFiles`, and `volumes` identify the logical `DeploymentService` that owns each input. `ociRuntime` and `cdiDevices` authorize only requirements already declared by that selected DOT deployment. These fields supply instance values and grants; they do not expand the DOT's declared configuration surface.

When a Docker Compose, Kubernetes, or Helm deployment contains several logical services with configuration, `deploymentInputs.workloads` is required for every configured service. The binding is deliberately instance-owned because native service names, labels, and containers can differ between deployment variants or environments:

```
deploymentInputs:
  workloads:
    - service: api
      compose:
        service: backend
```

The Kubernetes/Helm form uses exact-match standard labels and may select one container:

```
deploymentInputs:
  workloads:
    - service: api
      kubernetes:
        matchLabels:
          app.kubernetes.io/component: api
        container: api
```

Labels are combined with dotx's instance ownership selector. A live apply must resolve exactly one Deployment, StatefulSet, or DaemonSet, and an omitted container is accepted only when that workload has one container. Two logical services cannot target the same Compose service or the same Kubernetes workload/container pair. Plans expose the binding and secret identifiers, and locks cryptographically pin them; neither contains resolved secret values.

DOT definition versus DOT instance

One pinned DOT definition can appear more than once in a composition. Each entry has a distinct composition-local `name`, runtime identity, deployment inputs, and wires while retaining the same `dotId` and `dotVersion`:

```
dots:
  - name: site-a
    source: ./federation-agent
    dotId: d5000000-0000-4000-8000-000000000001
    dotVersion: 1.0.0
    deployment: docker
    deploymentInputs:
      environment:
        - {service: federation-agent, name: FEDERATION_NODE_ID, value: site-a}
  - name: site-b
    source: ./federation-agent
    dotId: d5000000-0000-4000-8000-000000000001
    dotVersion: 1.0.0
    deployment: docker
    deploymentInputs:
      environment:
        - {service: federation-agent, name: FEDERATION_NODE_ID, value: site-b}
```

Wires reference `site-a` or `site-b`, never the source path. Docker Image deployments receive deterministic per-instance container, DNS, network, and named-volume identities. Apply rejects repeated instances for deployment mechanisms whose manifest-owned resource names cannot yet be isolated safely.

The executable example is `DOT-Testing/composition/federation-peers/scenario.yaml`.

After defining the participants, wire and configure their selected Anchors.

Wiring, Materialization, and Security

Wires express composition topology outside every participating DOT. They refer to stable Anchor UUIDs, resolve deployable endpoints through service bindings, and materialize only values that the target service has declared as configurable.

Direct request/response wire

A wire uses instance names and stable Anchor UUIDs:

```
wires:
  - name: oauth-token-to-client
    from:
      instance: identity
      anchorId: a3000000-0000-4000-8000-000000000002
    to:
      instance: client
      anchorId: a5000000-0000-4000-8000-000000000001
    materializer:
      type: ENVIRONMENT_VARIABLE
      valueFrom: OPERATION_URL
      target:
        instance: client
        service: oauth-client
        environmentVariable: TOKEN_URL
```

Planning performs the following sequence:

1. Resolve the two pinned DOT instances.
2. Resolve both Anchor UUIDs and require complementary `PROVIDES`/`REQUIRES` roles.
3. Resolve the selected Contract elements.
4. Evaluate semantic, interaction, protocol, media, schema, and security compatibility.
5. Resolve the connection endpoint through the selected deployment's `anchorBindings`.
6. Verify that the target service declares `TOKEN_URL`.
7. Build `OPERATION_URL` from the runtime endpoint and the authoritative OpenAPI server/operation path.

The resulting URL is deployment data. It is never copied back into `dot.json` or the Anchor.

Cross-platform external gateway

A wire whose participants use different runtime platforms must declare a reviewed network adapter. The first supported adapter represents a gateway that the operator provisions outside `dotx`:

```
wires:
```

```
  - name: identity-discovery-to-proxy
    from: {instance: identity, anchorId: a3000000-0000-4000-8000-000000000001}
    to: {instance: proxy, anchorId: aa100000-0000-4000-8000-000000000002}
    network:
      adapter: EXTERNAL_GATEWAY
      host: identity-gateway.example.net
      port: 8080
```

The Contract and bound service port remain authoritative for protocol and interaction semantics. The binding replaces only the deployment-specific host and port used by materializers. The plan, lock, and private lifecycle state record `ownership=EXTERNAL` and `cleanup=NONE`; apply, resume, status, and destroy never claim to create, inspect, or remove the gateway. Cross-platform wires without a binding, same-platform wires with a redundant binding, unknown adapters, URL-shaped hosts, and invalid ports fail before runtime mutation.

Composition-owned gateways, ingresses, tunnels, and brokers require dedicated lifecycle adapters and remain unsupported.

A wire and a materializer are intentionally separate. The wire says *these Anchors are connected*; the materializer says *write this derived value into this declared consumer setting*. A wire needs no materializer when the runtime already provides discovery, a service mesh performs routing, both participants use a fixed broker declared elsewhere, or the composition is documenting an Anchor relation without an executable handoff. Apply still requires every actually required service configuration value to have a default or explicit input.

For the example above, the human-readable plan makes every resolution step inspectable before apply:

```
oauth-token-to-client:
  identity/a3000000-0000-4000-8000-000000000002
    -> client/a5000000-0000-4000-8000-000000000001 [COMPATIBLE]
  endpoint: keycloak:8080
    (HTTP, instance=identity service=keycloak port=http)
  contract: c3000000-0000-4000-8000-000000000002
    selector=#/paths/~1realms~1dotx~1protocol~1openid-connect~1token/post
  materializer: OPERATION_URL -> client/oauth-client env:TOKEN_URL
```

The same plan with `--json` is the machine-readable automation output. It includes stable instance and Anchor references, compatibility reasons, the selected Contract, resolved endpoint, derived operation/channel value, materializer targets, security status, and exposed-Anchor endpoints. Consumers should parse JSON rather than scrape the human-readable form.

Brokered wire

For messaging, the semantic provider, semantic consumer, and connection endpoint may be three different instances. Select the broker explicitly:

```
wires:
```

```
  - name: openc2-command-via-mqtt
    from:
      instance: interpreter
      anchorId: a6000000-0000-4000-8000-000000000003
    to:
      instance: actuator
      anchorId: a6000000-0000-4000-8000-000000000006
    endpoint:
      instance: broker
      service: mqtt
      port: mqtt
    materializers:
      - type: ENVIRONMENT_VARIABLE
        valueFrom: ENDPOINT_HOST
        target:
          instance: actuator
          service: openc2-actuator
          environmentVariable: MQTT_BROKER_HOST
      - type: ENVIRONMENT_VARIABLE
        valueFrom: ENDPOINT_PORT
        target:
          instance: actuator
          service: openc2-actuator
          environmentVariable: MQTT_BROKER_PORT
```

The AsyncAPI operations and message schema stay in the Contract. The broker's host and port stay in the resolved plan. If a channel has a templated address, the wire supplies only values for parameters declared by the Contract.

Materializers

Materializers are explicit adapters between a resolved endpoint and a configuration surface already declared by a service.

Type	Behavior
`ENVIRONMENT_VARIABLE`	Writes one resolved scalar to a declared environment variable.
`CONFIG_FILE`	Mounts a declared file and optionally replaces one YAML/JSON value selected by an RFC 6901 JSON Pointer.

Supported `valueFrom` values are:

Value	Result
`ENDPOINT_HOST`	Resolved service host only.
`ENDPOINT_PORT`	Resolved numeric service port only.
`ENDPOINT_HOST_PORT`	`host:port`.
`ENDPOINT_URL`	`scheme://host:port`, using the bound port protocol.

Value	Result
`OPERATION_URL`	Endpoint URL plus the authoritative OpenAPI server base and operation path.
`CHANNEL_ADDRESS`	Address of the authoritative AsyncAPI Channel Object.

An optional RFC 6570 `uriTemplate` can derive an application-specific URI from one resolved scalar. It is explicit composition behavior, not an Anchor or Contract fact.

Configuration-file materialization

A `CONFIG_FILE` materializer always names an operator-controlled source and a file path already declared by the target service. Without `valueFrom`, the source is supplied unchanged. With `valueFrom`, the source must be YAML or JSON and `jsonPointer` selects the scalar to replace:

```
materializer:
  type: CONFIG_FILE
  source: ./prometheus/prometheus.yml
  valueFrom: ENDPOINT_HOST_PORT
  target:
    instance: prometheus
    service: prometheus
    path: /etc/prometheus/prometheus.yml
    jsonPointer: /scrape_configs/0/static_configs/0/targets/0
```

This produces a deployable Prometheus configuration without putting the provider's runtime address in either DOT. The corresponding executable fixture is `DOT-Testing/composition/poc4-continuous-monitoring/scenario.yaml`.

URI-template derivation

Use `uriTemplate` only when the application needs an explicit deterministic derivation that is not already the selected Contract operation URL:

```
materializer:
  type: ENVIRONMENT_VARIABLE
  valueFrom: ENDPOINT_URL
  uriTemplate: "{+value}/realms/drax"
  target:
    instance: proxy
    service: proxy-xapp
    environmentVariable: OIDC_ISSUER
```

The RFC 6570 variable is named `value`. The template belongs to the composition because it adapts one deployment to another; it does not amend the Contract or redefine the Anchor.

Materialization fails when the target configuration is undeclared, the Contract path/channel remains unresolved, alternative server paths are ambiguous, two wires produce conflicting values

for one target, or the endpoint and target are on different runtime platforms without an explicit network binding.

Path and channel parameters

Only deployment-specific scalar values belong in the wire:

```
parameters:
  realm:
    value: drax
```

The Contract remains authoritative for the parameter name, location, requirement, schema, default, and serialization rules. Current composition bindings cover OpenAPI path parameters and AsyncAPI channel parameters. Query/header/cookie parameters, complex serialization, and runtime-derived parameter values require explicit future adapters.

Security and secrets

Contract security remains authoritative. A wire may demonstrate how one declared scheme is satisfied by deployment configuration:

```
security:
  bindings:
    - scheme: bearerAuth
      mechanism: https://www.rfc-editor.org/rfc/rfc6749#section-4.4
      participant: {instance: client, service: oauth-client}
      parameters:
        tokenUrl: {environmentVariable: TOKEN_URL}
        clientId: {environmentVariable: CLIENT_ID}
        clientSecret: {environmentVariable: CLIENT_SECRET}
```

Sensitive inputs are references, never secret values. They are scoped to the DOT instance whose selected deployment consumes them:

```
dots:
  - name: client
    source: repository://d3a00000-0000-4000-8000-000000000003@1.0.0
    dotId: d3a00000-0000-4000-8000-000000000003
    dotVersion: 1.0.0
    deployment: kubernetes
    deploymentInputs:
      environment:
        - name: CLIENT_SECRET
          secretRef:
            store: kubernetes
            name: oauth-client
            key: client-secret
```

Plans and composition state keep only the reference. Docker deployments can resolve `store: environment` at final handoff; Kubernetes and Helm preserve `store: kubernetes` as `secretKeyRef` without reading the Secret. A `CONFIGURED` plan proves configuration completeness, not token validity, issuer trust, mTLS, workload identity, or remote attestation.

Exposed Anchors

A composition may re-export a member Anchor under a composition-local name:

```
exposedAnchors:
  - name: protected-metadata-through-gateway
    target:
      instance: gateway
      anchorId: a4000000-0000-4000-8000-000000000003
```

At apply time, an operator may request a local forward:

```
dotx compose apply scenario.yaml \
  --forward protected-metadata-through-gateway=19080
```

The exposure name belongs to the composition boundary. It does not rename or replace the member Anchor UUID.

Before runtime mutation, follow the validation and lifecycle workflow.

Composition Lifecycle and Design Guidance

The lifecycle deliberately separates reproducible input locking, model validation, deterministic planning, runtime mutation, recorded state, and active acceptance evidence. This keeps review and CI read-only while making execution recoverable and auditable.

Validation, planning, and lifecycle

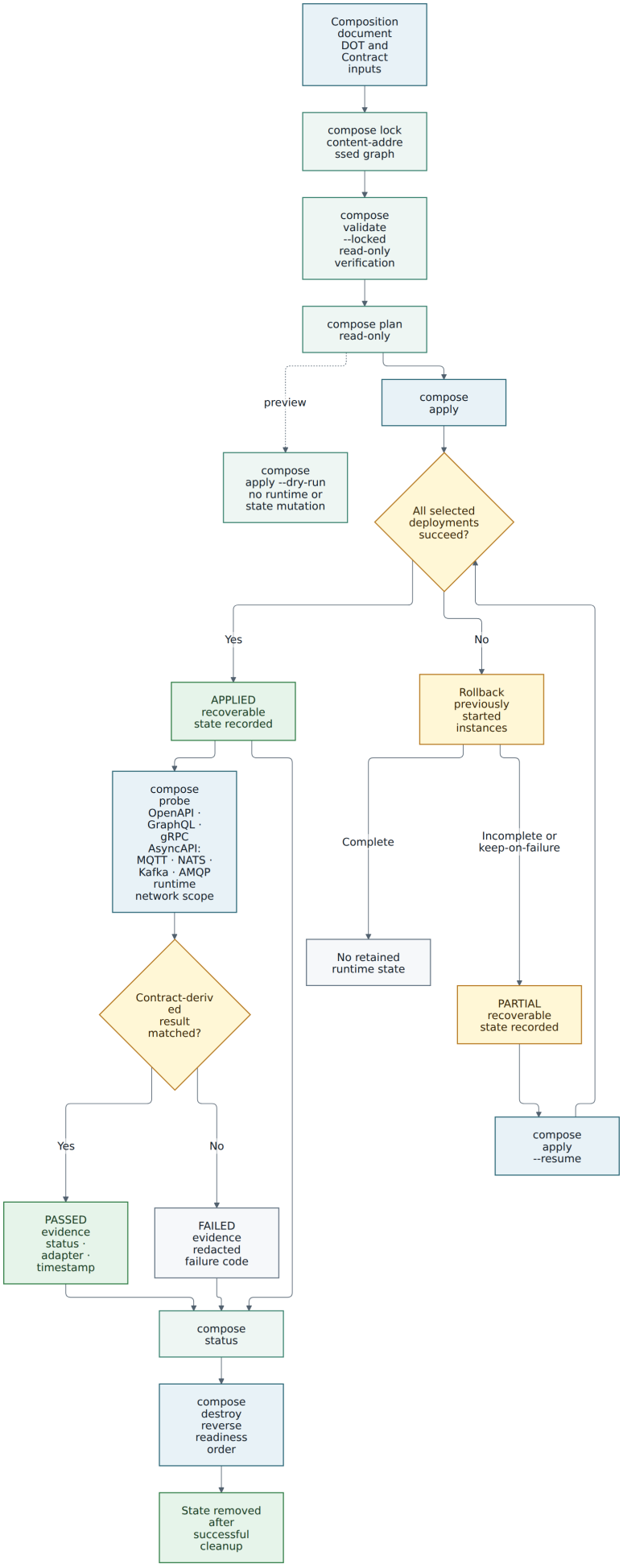


Figure 7. Composition lifecycle

What each command proves

Command	Proves	Does not prove	Mutates runtime/state
`compose lock`	The exact composition, selected DOT definitions and deployments, Contracts, configuration inputs, and portable resolved plan are content-addressed.	Runtime availability, credentials, or policy acceptance.	Writes only the requested lock file; `--check` is read-only
`compose validate`	The strict document shape, references, selected deployments, Anchors, Contracts, compatibility evidence, and materializer declarations resolve.	The exact runtime actions or live availability.	No
`compose plan`	Everything validated above plus the deterministic instance order, runtime identities, endpoints, Contract standard/format and participant interaction, derived values, security status, and materializers.	That the planned hosts are reachable or credentials work.	No
`compose apply --dry-run`	The composition reaches lifecycle preparation and can render supported deployment actions without persisting composition state.	Successful deployment or application-level traffic.	No
`compose apply`	The supported deployment adapters completed and recoverable state was recorded, subject to rollback behavior.	End-to-end Contract behavior unless separately probed.	Yes
`compose upgrade --dry-run`	The exact current and target locks resolve, the target SemVer increases, and a deterministic per-instance `ADD`/`UPDATE`/`REMOVE`/`UNCHANGED` diff passes runtime, storage, ownership, and recovery preflights.	That an application-specific persistent-data migration is safe.	No

Command	Proves	Does not prove	Mutates runtime/state
`compose upgrade`	The target revision was applied through a checkpointed transaction and the last-known-good control-plane revision remained available until promotion.	Reversal of application data writes; stateful mutations are rejected.	Yes; failures automatically restore the verified previous revision, while interrupted transactions require `--resume` or `--rollback`
`compose probe`	A selected safe OpenAPI operation, GraphQL lifecycle, finite side-effect-free gRPC method, or correlated AsyncAPI MQTT/NATS/Kafka/AMQP message is observable from the composition's shared Docker network or Kubernetes namespace. GraphQL mutations/subscriptions include mandatory verification or trigger and cleanup.	Complete application semantics, unsafe or unbounded gRPC, other AsyncAPI protocols, multi-scheme authentication, authorization-policy quality, or unprobed operations.	Creates and removes an ephemeral probe runner without changing member workloads; writes redacted probe evidence to composition state
`compose status`	Native status for recorded instances, wire/exposure configuration evidence, and the latest recorded active-probe outcomes.	Current reachability after the probe timestamp, authentication mechanisms not explicitly identified by the evidence, authorization semantics, response bodies, or policy compliance.	No runtime mutation; reads state while holding the lifecycle lock
`compose destroy`	Owned instances were destroyed in reverse order and successful cleanup removed composition state.	Cleanup of resources not owned or recorded by this composition.	Yes

`validate` answers *is this composition internally resolvable?* `plan` answers *what exactly would be connected and materialized?* Keeping both commands read-only makes them suitable for review and CI.

`lock` makes those answers reproducible. Its JSON output preserves the original `repository://` and relative references while recording SHA-256 digests for the composition, every resolved `dot.json`, every supplied Contract, configuration inputs, and the portable resolved plan. For an explicit remote repository it also records the normalized repository identity and exact commit. It never records secret values, runtime addresses, or checkout-local absolute paths. Validate, plan, and apply

verify an adjacent lock automatically; `--locked` additionally requires it, and `--lock <path>` selects an explicit file. Any mismatch fails before runtime mutation.

`probe` deliberately reuses the interaction selected by the wire instead of introducing health-check fields in an Anchor or DOT. For OpenAPI, it derives the request from the authoritative operation, accepts only `GET`, `HEAD`, and `OPTIONS`, and requires one fully bound path. The OAuth 2.0 adapter reuses one configured RFC 6749 client-credentials binding that satisfies a complete, single-scheme OpenAPI Security Requirement alternative. It resolves the provider-derived token URL, client ID, scopes, and client-secret reference at execution time, then requires a Contract-declared `2xx` response from the protected operation. Kubernetes Secret values remain `secretKeyRef` references; Docker secrets are supplied only through the runner environment.

For GraphQL, the composition supplies the deployment-specific HTTP path and a non-sensitive operation document with optional operation name and variables. Before planning succeeds, `dotx` parses the authoritative SDL, validates every document and variable set, requires exactly one direct top-level field, and requires that field to equal the Anchor-selected coordinate. It never guesses `/graphql`. Queries use HTTP directly. Mutations additionally declare an observable verification query and cleanup mutation. Subscriptions use the GraphQL-over-SSE protocol on a connection established before a separate trigger mutation and finish with a cleanup mutation. Every phase must return HTTP `2xx` without GraphQL errors; the expectation is evaluated against the query response, mutation verification result, or delivered subscription event.

For gRPC, the composition supplies one protobuf-JSON `request` for unary and server-streaming methods, or an ordered `requests` sequence for client and bidirectional streams. The selected Proto3 RPC must declare the standard `NO_SIDE_EFFECTS` idempotency level and be discoverable through explicitly enabled server reflection. Every streaming fixture declares a positive exact `responseCount`; an optional expectation uses `ALL`, `ANY`, `FIRST`, or `LAST` matching. Requests are checked against the authoritative input descriptor before the pinned `grpcurl` runner starts, then every returned document is checked against the authoritative output descriptor. Unsafe, unbounded, wrong- count, or descriptor-incompatible calls fail closed.

For AsyncAPI, the active adapters require a provider `send` operation, an explicit MQTT/MQTT5, NATS, Kafka, or AMQP channel binding, one concrete destination, and a JSON-Schema message payload. The composition declares the message and RFC 6901 correlation pointer; MQTT additionally declares QoS. The Kafka adapter honors the official binding-level `topic` override and restricts topics to the Kafka-supported ASCII alphanumeric, dot, underscore, and hyphen syntax. The digest-pinned official Mosquitto or NATS client subscribes before publishing a fresh UUID-correlated non-retained message. Kafka creates a unique consumer group, waits for its topic assignment before publishing the exact correlated record through the official Apache Kafka JVM image, and removes the group after the bounded check. Kafka records remain subject to broker/topic retention and are never represented as transient. The AMQP 0-9-1 adapter applies the official 0.3.0 channel and operation bindings, declares the exact direct/topic/fanout exchange, binds an exclusive auto-delete queue before publishing, acknowledges the correlated delivery, and closes the connection to remove the queue. It accepts credentials only as a password secret reference. Queue-bound channels, headers exchanges, AMQPS, AMQP 1.0, a binding/protocol mismatch, or MQTT-only QoS on NATS/Kafka/AMQP fail closed.

All adapters execute digest-pinned runners inside the common Docker network or Kubernetes namespace. Runner output is capped at 1 MiB before parsing. Secrets and bearer tokens never

appear in the argument vector or state. Contractless, unresolved, unsupported-protocol, and cross-platform wires are reported as unsupported rather than inferred healthy. Diagnostics, documents, variables, requests, messages, and response bodies are not persisted; state contains only normalized identifiers, adapter, result metadata, failure code, and timestamp.

The live contract-probe fixture runs a real GraphQL.js producer with distinct GraphQL-over-SSE connections, a reflected `grpc-js` server implementing all four RPC shapes, Eclipse Mosquitto, NATS, Apache Kafka, and RabbitMQ. It demonstrates the full supported boundary; it does not turn those test services or interaction fixtures into reusable DOT metadata.

```
# Composition-level probe input; the DOT and Anchor still contain only the
# authoritative Contract reference and selector Query.health.
probe:
  graphql:
    path: /graphql
    operationName: Health
    document: |
      query Health {
        health
      }
    expect: {pointer: /data/health, equals: true}
```

```
# Finite bidirectional gRPC probe. The selected Proto method in the Contract
# declares stream input/output and NO_SIDE_EFFECTS; these fields supply only
# instance-specific test messages and acceptance criteria.
probe:
  grpc:
    plaintext: true
    serverReflection: true
    requests:
      - {component: contract-provider, count: 1}
      - {component: contract-consumer, count: 1}
    responseCount: 2
    responseMatch: LAST
    expect: {pointer: /sequence, equals: 2}
```

```
# NATS transport delivery. The selected AsyncAPI channel owns the subject,
# NATS binding, message schema, and send/receive semantics.
probe:
  asyncapi:
    protocol: NATS
    correlationPointer: /id
    message: {id: replaced-at-runtime, severity: critical}
```

```
# Kafka retained-stream delivery. The selected AsyncAPI Kafka 0.5.0 channel
# binding owns the physical topic override and the message schema.
```

```
probe:
  asyncapi:
    protocol: KAFKA
    correlationPointer: /id
    message: {id: replaced-at-runtime, severity: critical}
```

```
# Emit the canonical strict Draft 2020-12 schema
dotx compose schema > composition-v1.schema.json

# Generate once, commit, and verify without rewriting
dotx compose lock scenario.yaml
dotx compose lock scenario.yaml --check

# Structure, references, selectors, compatibility, and materializer declarations
dotx compose validate scenario.yaml --locked

# Deterministic resolved topology; no runtime mutation
dotx compose plan scenario.yaml --locked
dotx compose plan scenario.yaml --locked --json

# Resolve the catalogue from one immutable remote Git revision
dotx compose plan scenario.yaml --locked --json \
  --repository https://github.com/MARE-EU-Project/DOT-Repository.git \
  --git-ref <40-character-commit>

# Render runtime actions without state or runtime mutation
dotx compose apply scenario.yaml --locked --dry-run

# Execute, inspect recorded evidence, and clean up owned resources
dotx compose apply scenario.yaml --locked
dotx compose probe scenario.yaml --locked --wire oidc-provider-to-gateway
dotx compose probe scenario.yaml --locked --wire protected-gateway-to-client
dotx compose status scenario.yaml
dotx compose destroy scenario.yaml

# Review and execute a locked upgrade
dotx compose upgrade scenario-v2.yaml \
  --from scenario-v1.yaml \
  --from-lock scenario-v1.lock.json \
  --locked --dry-run --json
dotx compose upgrade scenario-v2.yaml \
  --from scenario-v1.yaml \
  --from-lock scenario-v1.lock.json \
  --locked
```

Apply records each pending instance before runtime mutation, rolls back earlier instances on failure, and preserves recoverable state when rollback is incomplete or `--keep-on-failure` is selected. `apply --resume` requires an identical composition source, resolved plan, runtime options, inputs, ordering, and previously applied runtime state. Resume also reconciles an interrupted

composition-network creation: it reuses the correctly owned network or creates the still-missing network before continuing. Destroy checkpoints every successfully removed instance, so a retry targets only cleanup that did not finish.

Upgrade is a separate lifecycle; `apply` never mutates an already recorded plan. Upgrade requires the exact previous and target locks, the same composition UUID, and a strictly greater target SemVer. Existing instance names must retain their platform and deployment mechanism. Docker image, Docker Compose, Kubernetes manifest, Helm, Vagrantfile, and WASM module instances use their stable managed identities. The private transaction journal stores only paths, identifiers, phases, and digests—not resolved secrets or input payloads. Stateful and host-bound mutations fail before the first runtime action because restoring a deployment specification cannot undo workload data changes.

The deterministic recovery profile injects an interruption immediately before and after network creation, instance deployment, instance destruction, and network removal. It also verifies lock contention, lock release after a panic, partial cleanup, idempotent resume, and fail-closed plan/runtime drift:

```
task testing:compose:lifecycle-recovery
task testing:compose:upgrade
```

The task writes revision-pinned, secret-free JSON evidence under `DOT-Testing/e2e/target/` and is enforced by the Composition lifecycle recovery workflow.

Apply, probe, status, and destroy acquire a private per-composition process lock. A concurrent lifecycle command fails immediately rather than racing local state or runtime mutations.

`compose status` reports:

- the recorded phase and native runtime status of each DOT instance;
- recorded wire compatibility, security configuration, selected endpoint, and materializer count; and
- recorded exposed-Anchor bindings; and
- the latest redacted active Contract-probe result for each probed wire.

Wire and exposure entries remain configuration evidence. Only entries under active Contract-probe evidence demonstrate a runtime request and declared OpenAPI response match, bounded by their timestamp and adapter scope.

Fail-closed rules

Apply rejects:

- `UNKNOWN` wire compatibility;
- missing, duplicate, or mismatched IDs and versions;
- ambiguous deployments, bindings, services, ports, or OpenAPI server paths;

- unresolved OpenAPI paths or AsyncAPI channel templates;
- undeclared materializer targets and conflicting target values;
- readiness dependency cycles;
- unsupported secret-store/runtime combinations;
- ungranted OCI capabilities, host namespaces, volumes, or CDI devices;
- repeated DOT instances on deployment mechanisms that cannot isolate their resource names; and
- materialized cross-platform wires without an explicit network binding.

Planning may retain `UNKNOWN` to explain incomplete evidence. Execution never silently upgrades `UNKNOWN` to compatible.

What does not belong in an Anchor

Do not add these as optional Anchor fields:

- hosts, URLs, numeric ports, broker addresses, or Kubernetes names;
- credentials, tokens, certificates, or secret values;
- Camel/Kamelet route syntax;
- transformation logic;
- placement, slice, tenant, trust-zone, latency, or availability policy;
- current health or live authorization state;
- host paths or operator device names; or
- orchestration dependencies.

Use Contracts for interaction semantics, service bindings for deployable ports, composition inputs for operator-controlled configuration, wires for topology, materializers or mediator DOTs for explicit adaptation, and policy engines for deployment/security decisions.

Choosing an extension point

Requirement	Preferred representation
New API/message operation	Add or update the authoritative Contract and select it from an Anchor.
Consumer needs a provider address	Wire materializer targeting declared service configuration.
Payload/protocol transformation	Explicit mediator DOT with two Contracts and two Anchors.
Startup/readiness ordering	Composition `dependsOn`.
Secret or certificate	Reference-only deployment input and platform/external secret provider.

Requirement	Preferred representation
Privileged runtime resource	Typed deployment requirement plus exact composition grant.
Broker service	Explicit wire `endpoint` selecting the broker service port.
Network attachment, eBPF/XDP, WASM import/export, or lifecycle event	Separately reviewed polymorphic binding type when a real use case requires it; do not grow `Anchor` with nullable fields.

6G security composition patterns

The same model supports security-function composition without baking one orchestrator's topology language into every DOT:

- **Zero-trust API chain:** identity-provider and policy-decision DOTs expose token, discovery, and authorization Anchors; an API gateway consumes them and exposes a protected application Anchor. Contracts carry OAuth/OpenID Connect and application semantics, while the composition supplies client-specific secret references and issuer URLs.
- **Closed-loop detection and response:** telemetry, inference, CACAO interpretation, and OpenC2 actuation remain separate DOTs. HTTP and MQTT Contracts define the boundaries; the wire selects the broker endpoint and supplies only deployment-specific channel values.
- **Multi-domain federation:** peer DOT instances can have reciprocal wires while `dependsOn` contains only real readiness dependencies. Trust-domain, roaming, and federation policy belongs in policy inputs or a dedicated policy DOT, not in the Anchor.
- **Near-real-time RAN control:** E2 or management APIs are Contract-backed Anchors. Host networking, accelerators, eBPF/XDP attachments, and radio-device access are explicit deployment requirements and grants, never inferred from a wire.
- **Observability and digital twins:** metrics, logs, events, and integrity APIs are distinct Anchors. A storage or message broker is selected explicitly as a connection endpoint; a shared host path is not treated as a portable integration boundary.

For cross-domain 6G deployments, a successful plan proves model consistency, not trust. Admission policy must still evaluate publisher identity, artifact provenance, attestation, slice/tenant placement, data-sovereignty constraints, and least-privilege grants before execution. Runtime acceptance should then probe the selected Contract operation over the actual network path.

Troubleshooting

Symptom	What to check
Anchor UUID is not found	The wire references the pinned DOT version, the UUID is stable, and the Anchor is present in that version's `dot.json`.
Contract or selector cannot be resolved	The composition maps the exact Contract UUID to a readable source, the source document's `id` matches it, and the selector uses the standard-specific syntax.

Symptom	What to check
Anchor has no deployable endpoint	The selected deployment contains exactly one service binding for the Anchor, and its named port exists on that same service.
Compatibility is `UNKNOWN`	Inspect `compatibilityReasons` in `compose plan --json`; add authoritative Contract evidence or use an explicit mediator. Do not override the result.
Materializer target is rejected	The target service declares the environment variable or config-file path, the JSON Pointer is valid, and no other wire writes a conflicting value.
Security remains unresolved	Bind a Contract-required supported scheme, declare all participant variables, and supply sensitive values by `secretRef` rather than plaintext.
Apply reports the composition is busy	Another apply, status, or destroy holds the per-composition lock. Let that command finish; do not delete the lock while a lifecycle process is running.
Status shows a configured wire but no active evidence	Run `dotx compose probe scenario.yaml --wire <wire>`. If the wire is skipped, its operation requires a dedicated adapter or application-level acceptance test. Configuration evidence alone is not reachability evidence.
An OAuth probe reports `AUTHENTICATION_ERROR`	Check the bound token URL and client ID, the referenced Secret name/key or environment-store source, required scopes, issuer reachability, and client authorization. Dotx deliberately does not persist the token endpoint's body.

Executable examples

- `DOT-Testing/conformance/anchors/` covers all four Contract standards, contractless Anchors, validation failures, bindings, and compatibility.
- `DOT-Testing/e2e/api-gateway-oidc/` is the canonical authenticated request/response lifecycle example.
- `DOT-Testing/composition/poc6-closed-loop/` demonstrates an explicit MQTT broker endpoint and OpenC2/AsyncAPI operations.
- `DOT-Testing/composition/federation-peers/` demonstrates reciprocal wires without false readiness cycles.
- `DOT-Testing/e2e/poc1-ric-aaa/` demonstrates identity, OPA authorization, forwarding, metrics feedback, and deny-list update across seven wires.
- `DOT-Testing/interoperability/pocs.yaml` is the evidence ledger for all eleven MARE PoCs and records every remaining acceptance gap.

Run the repository gate with:

```
task testing:conformance
```

The source-adapted E2E tests prove only their documented boundaries. They do not claim partner-runtime, live-RIC, production-network, trust-policy, or SFC acceptance unless that exact behavior is asserted by the scenario.

Verification and examples

DOT-Design keeps model correctness, resolved planning, deployment dry-runs, and live behavioral acceptance as separate evidence levels. A passing lower level never implies a higher one.

Evidence model

Level	Demonstrates	Does not demonstrate
Model and schema conformance	`dot.json` shape, Bean Validation constraints, stable references, selectors, service-port bindings, and generated-model compatibility	Contract payload resolution or multi-DOT topology
Contract resolution and compatibility	Selected operations exist and normalized semantic, interaction, protocol, media, schema, and security facts can be evaluated conservatively	Runtime reachability or valid credentials
Composition validation and plan	Pinned DOTs, wires, endpoints, materializers, parameters, grants, and readiness ordering resolve deterministically	Successful deployment
Lifecycle dry-run	The reference executor accepts the supported profile and renders runtime actions without writing composition state	Workload startup or traffic
Source-adapted E2E	Pinned upstream logic or artifacts execute behind explicit test adapters and satisfy the assertions documented by that scenario	Unmodified partner runtime, production infrastructure, or full PoC acceptance
Live production acceptance	The intended workloads, network, identity, policy, and Contract operations work in the target environment	Nothing beyond the probes and acceptance criteria actually executed

Primary verification commands

Command	Scope	External requirements
`mvn -pl DOT-Domain test`	Domain value objects, manifests, Anchors, and aggregate validation	Java and Maven only
`go -C DOT-CLI test ./…`	CLI parsing, validation, planning, deployment adapters, and lifecycle behavior	Go; individual tests use doubles where appropriate

Command	Scope	External requirements
`task testing:dots:repository`	Every `dot.json` discovered recursively below `DOT-Repository/DOTs/`, with the reusable profile for external/internal bundles that publish adjacent Contract evidence, ownership-path isolation, mandatory per-bundle README files, schema validity, CLI cross-references, and identity uniqueness	Go and the checked-out `DOT-CLI` and `DOT-Repository` revisions; repository validation uses its co-versioned `Schema/`, validates the authoritative `Skills/` catalogue, and resolves every DOT Skill UUID to one categorized capability
`task testing:dots:external:dry-run`	Reusable Keycloak/Prometheus DOTs plus runtime-only external Loki/Wireshark definitions: profile-appropriate validation, inspection, deployment dry-run, exact-version Helm rendering, expected service/workload shape, and pinned OCI image metadata	`dotx`, Helm, Python/PyYAML, Docker, and network access to the pinned public artifacts
`task testing:quickstart:dry-run`	Execute the command blocks included by Getting started: validate and render Prometheus, then check the locked gateway composition plan and dry-run	`dotx`, `jq`, initialized catalogue, and network access for official Contracts
`task testing:quickstart:live`	Run Prometheus at the default and overridden port, assert the build-information API/version, and verify container/network/volume cleanup on success and injected failure	Docker Compose, `dotx`, `curl`, `jq`, network access, and free host ports `18909`, `18919`, and `18929`; all example metrics are deleted
`task testing:dots:external:stress`	Repeated Linux deployment, readiness, HTTP/HTTPS access, destroy, and cleanup across two mechanisms per external product: three Helm/Compose pairs plus the Wireshark Docker-image/Compose pair; Keycloak and Prometheus additionally call the exact operations selected from their official Contracts	Docker, Minikube/Kubernetes, Helm, network access, and enough resources for Keycloak, Loki, Prometheus, and Wireshark; emits revision-pinned evidence
`task testing:compose:keycloak-prometheus-openapi-planning`	Reusable-profile validation and a locked three-DOT plan proving the stable Keycloak and Prometheus Anchors resolve against exact official OpenAPI payloads, compose compatibly, bind the Keycloak realm parameter, and materialize the expected consumer operation URLs	`dotx`, `jq`, and the checked-out repository/schema revisions; no live runtime

Command	Scope	External requirements
`task testing:compose:contract-standards`	Two real DOT definitions connected through OpenAPI 3.0/3.1/3.2 and AsyncAPI 3.0/3.1 YAML/JSON, Protocol Buffers (including gRPC service declarations), and GraphQL SDL; asserts selectors, participant interactions, endpoints, probe fixtures, and materializers across twenty-one wires	`dotx`, `jq`, and the checked-out repository/schema revisions
`task testing:compose:contract-probes:live`	A real producer and four brokers prove GraphQL query, mutation verification/cleanup, GraphQL-over-SSE delivery, all four finite side-effect-free gRPC shapes through reflection, correlated non-retained AsyncAPI MQTT/NATS delivery, correlation-scoped Kafka delivery under broker retention, and acknowledged AMQP 0-9-1 delivery through an exclusive probe queue; emits redacted evidence	Docker, network access for pinned images, and the checked-out `dotx`
`task testing:compose:acceptance`	Authoritative `v1` command/schema parity, every committed lock, twenty-one Contract wires, contractless compatibility, all six runtime dry-runs, service-scoped configuration, recovery/status/destroy, and focused fail-closed controls; emits revision-pinned JSON evidence	A globally installed `dotx` from the checked-out CLI revision, Go, Task, and fixture-specific dry-run CLIs
`task testing:contracts:standards`	Extracts the twelve distinct Contract payloads and validates them with dated official OAI schemas, the official AsyncAPI CLI, `protoc`, and GraphQL.js; each family also has an invalid control which must be rejected	Python with PyYAML/jsonschema, Node.js, network access for pinned validators, and `protoc`
`task testing:compose:runtime-configuration`	The same catalogued DOT receives declared environment and configuration-file inputs through Docker image, Docker Compose, Kubernetes manifest, Helm chart, Vagrantfile, and Wasmtime/WASI dry-run adapters	`dotx`, `jq`, and runtime CLIs in dry-run rendering mode
`task testing:compose:service-workloads`	A two-service DOT receives isolated defaults, overrides, configuration files, and secret references through exact Compose, Kubernetes, and Helm workload bindings	`dotx`, `jq`, and runtime CLIs in dry-run rendering mode

Command	Scope	External requirements
`task testing:compose:service-workloads:live`	Two concurrent copies of the same multi-service workload prove exact environment, secret, config-file, and volume isolation plus readiness, status, resume, selective destroy, surviving-peer health, and cleanup across Docker Compose, Kubernetes, and Helm	Docker Compose, Minikube/Kubernetes, Helm, and network access for runtime images; emits redacted revision-pinned evidence
`task testing:compose:runtime-isolation:live`	Two locked instances of one Vagrant DOT and one WASI CLI DOT prove distinct runtime identities, concurrent state, status, selective destroy, peer survival, and cleanup. Repeated fixed-port WASI HTTP fails before mutation.	Vagrant 2.4.9 with libvirt/KVM and Wasmtime 49.0.2; emits redacted, revision-pinned evidence
`task testing:cli:coverage`	Complete DOT-CLI test suite plus machine-readable total and high-value package statement-coverage profiles	Go; writes reports below `DOT-Testing/e2e/target/`
`cd DOT-CLI && task ci:security`	Scans the Go module dependency graph and reachable code against the current OSV vulnerability database with the exact pinned scanner used by CI	Go and network access; intentionally separate from deterministic `task ci`
`task testing:runtime:kubernetes-helm:live`	Real Kubernetes-manifest and Helm-chart deploy, rollout readiness, status, service-scoped environment materialization, and destroy lifecycle	Docker, Minikube, Kubernetes, Helm, and network access for runtime images
`bash DOT-Testing/runtime/wasm/live.sh`	Real Wasmtime lifecycle for WASI CLI; CI additionally builds the pinned Bytecode Alliance `wasi:http/proxy` sample and proves an HTTP response	`dotx`, Wasmtime, and the optional prebuilt HTTP component path
`task testing:conformance`	Cross-language Anchor and composition gate, including all deterministic PoC fixtures and lifecycle dry-runs	Java, Maven, Go, Task, and fixture-specific local tools
`mvn clean install`	Full Java domain build, schema generation, coverage gates, and SBOMs	Java and Maven only
`task testing:e2e:<scenario>`	Scenario-specific live behavior	Read that scenario's README first; commonly Docker and network access

Test asset layout

```
DOT-Testing/
 ├──── acceptance/        versioned, implementation-neutral consumer profiles
 ├──── conformance/       deterministic valid and invalid DOT fixtures
```

```
├──── composition/       read-only validate, plan, and dry-run scenarios
├──── coverage/          executable code-coverage ratchets
├──── e2e/               opt-in deployable behavioral scenarios
├──── interoperability/  revision-pinned evidence and remaining-gap ledger
└──── runtime/           isolated native-runtime lifecycle harnesses
```

`testing` is repository-level infrastructure, not a Maven module and not a deployable DOT. Authoritative test-only definitions live under `DOT-Repository/DOTs/testing/` and are selected from compositions with `repository://<dot-id>@<version>`. Intentionally invalid conformance inputs stay under `DOT-Testing/conformance/`; they must never be admitted to the repository catalogue. Generated secrets, logs, reports, coverage profiles, and materialized configuration belong under scenario-local `target/` directories and must not be committed.

The focused `task testing:anchors:acceptance` gate verifies the versioned DOT Anchor consumer profile. It resolves pinned test definitions from `DOT-Repository`, verifies the composition lock, and asserts the complete cross-standard plan without a deployed workload.

The separate `task testing:compose:acceptance` gate freezes the complete `dot.mare6g.eu/v1` production profile. Its machine-readable source is `DOT-Testing/acceptance/composition/v1/profile.json`; the gate binds that source to CLI help, the tracked schema, repository locks, positive evidence, and negative fail-closed behavior. Runtime rows distinguish underlying single-DOT live lifecycle proof from multi-DOT composition lifecycle proof.

The focused `task testing:compose:contract-standards` gate complements that API profile with an executable composition matrix. It proves all six supported standard/format representations between the same provider and consumer DOTs; it does not promote planning evidence to live protocol or application acceptance.

The separate `task testing:contracts:standards` gate validates those payloads outside the DOT resolvers with standards-native tooling. The positive fixtures must pass and deliberately malformed OpenAPI, AsyncAPI, Protobuf, and GraphQL controls must fail. This catches both invalid examples and accidentally permissive validator invocations.

The runtime-configuration gate separately proves that selecting another executable deployment does not discard a service's declared environment or configuration-file surface. Each adapter is exercised first with its declared `defaultValue` and `defaultContent`, then with explicit composition overrides; the gate validates the cached default document, resolved service id, override values, and runtime-specific dry-run. Runtime-native storage, secret, privilege, and device semantics retain dedicated tests rather than being flattened into a misleading universal volume abstraction.

The multi-service live profile is the higher evidence layer for the exact workload bindings. Compose projects and their named volumes are keyed by the composition runtime identity; Kubernetes repetitions use separate namespaces; Helm repetitions use identity-derived release names in one namespace. The profile verifies that destroying one copy leaves the other healthy and that dotx-created ConfigMaps/Secrets are cleaned without deleting referenced operator-owned Secrets. The tracked Helm chart is packaged from the pinned checkout, and its SHA-256 is included in the redacted evidence artifact.

Coverage claims and gates

Coverage is reported by evidence dimension; the project does not collapse different meanings of "covered" into one percentage:

Dimension	Gate	Interpretation
Repository inventory	`dotx validate DOT-Repository`, `dotx repository audit`, and the DOT repository conformance workflow	Must be 100%: every filesystem definition is discovered, parsed, schema-valid, cross-reference-valid, uniquely identified, and stored in a supported category; no deployable copy may remain outside `DOT-Repository`.
CLI statements	`task testing:cli:coverage`	The whole CLI and every package containing executable statements must retain 100% statement coverage. The constant-only `feature/sources` package has no testable statements. This is a code-execution metric, not proof that a workload or security control works.
Java instructions and branches	`mvn verify` through JaCoCo 0.8.15	The Java domain module must retain 100% instruction and branch coverage. JaCoCo HTML/XML/CSV reports are uploaded with the Java build evidence.
Native runtime lifecycle	WebAssembly and Kubernetes/Helm runtime E2E workflows	Proves real Wasmtime CLI and HTTP behavior plus Minikube manifest/Helm lifecycle. It does not imply that every external DOT workload is accepted.
Composition scenarios	`task testing:e2e:all:dry-run`	Every listed deterministic scenario must validate, plan, and complete its supported lifecycle dry-run.
PoC interoperability	`DOT-Testing/interoperability/pocs.yaml` plus scenario-specific live workflows	Each of the eleven PoCs has traceable evidence, while remaining adapters, unavailable partner components, and unproved production behavior stay explicit.

The repository validation gate fails on duplicate identities, mis-scoped definitions, invalid DOTs, invalid Skill/category assets, or unresolved DOT Skill references. The workspace audit separately classifies uncatalogued, identical, divergent, and malformed definitions and fails on every deployable `dot.json` outside `DOT-Repository`. `DOT-Testing/conformance/` fixtures are reported but exempt because invalid definitions are their purpose. During the staged migration, run:

```
dotx repository audit --repository DOT-Repository --workspace . --json
```

The statement report deliberately has no artificial 100% threshold: a high line percentage cannot replace negative tests, lifecycle checks, live probes, or partner acceptance.

Canonical Anchor scenario

`DOT-Testing/e2e/api-gateway-oidc/` is the primary tutorial and lifecycle fixture. Its composition resolves three independently valid, test-scoped definitions from `DOT-Repository`:

- Keycloak provides OpenID Connect discovery and token operations;
- APISIX requires discovery information and provides a protected operation; and
- an OAuth client requires the token and protected operations.

The composition uses three OpenAPI Contracts, three wires, explicit security configuration, Kubernetes Secret references, configuration-file and environment materializers, readiness dependencies, and one exposed Anchor.

```
task testing:compose:keycloak-apisix-oauth
task testing:demo:keycloak-apisix-oauth:dry-run

# Opt-in live execution
task testing:demo:keycloak-apisix-oauth:live
```

Read DOT Anchors and Composition alongside `DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml`.

MARE PoC-derived coverage

The interoperability ledger pins the source revision used for every analysis, records the modeled interactions, identifies the executable fixtures, and keeps remaining acceptance gaps explicit. The current repository contains source-adapted evidence for all eleven PoCs; this is not a claim of unmodified partner-runtime acceptance.

PoC	Composition focus	Fixture
1	OIDC identity, OPA authorization, RIC forwarding, metrics feedback, and deny-list update	`DOT-Testing/composition/poc1-ric-aaa/`
2	Repeated Flower trainers over gRPC/Protobuf and exposed BentoML inference	`DOT-Testing/composition/poc2-federated-inference/`
3	Five-stage contractless JSON insider-threat analytics dataflow	`DOT-Testing/composition/poc3-insider-threat-pipeline/`

PoC	Composition focus	Fixture
4	Prometheus metrics, Elasticsearch logs, Grafana, and Kibana configuration	`DOT-Testing/composition/poc4-continuous-monitoring/`
5	Monitoring-to-analysis-to-assessment-to-mitigation security pipeline	`DOT-Testing/composition/poc5-security-pipeline/`
6	OpenAPI plus brokered AsyncAPI/MQTT, CACAO, OpenC2, and NWDAF remediation	`DOT-Testing/composition/poc6-closed-loop/`
7	Least-privilege packet capture, explicit CloudEvents mediation, and trust scoring	`DOT-Testing/composition/poc7-trust-scoring/`
8	cAdvisor/Prometheus observability and physical/twin integrity checks	`DOT-Testing/composition/poc8-ndt-observability/` and `poc8-twin-integrity/`
9	Amarisoft/Open5GS collection, InfluxDB storage, and data export	`DOT-Testing/composition/poc9-edge-ddos-collection/`
10	OIDC-protected capability exposure, SPARQL, OPA policy decisions, and observability	`DOT-Testing/composition/poc10-secure-capability-exposure/`
11	Reciprocal federation peers and local-core adapters with separate readiness edges	`DOT-Testing/composition/federation-peers/`

The authoritative coverage and gap list is `DOT-Testing/interoperability/pocs.yaml`. The official project descriptions are available from the MARE PoC catalogue.

Negative evidence is intentional

Rejected fixtures are part of the design evidence. They demonstrate that the model does not invent connectivity where the pinned source lacks a declared consumer input, does not treat stdout or a host path as a portable connection, and does not silently grant privileged runtime access.

Examples include:

- PoC7 rejects a collector-to-scorer wire when the collector declares no such Anchor or configuration input;
- PoC8 rejects an undeclared Prometheus-to-NDT handoff; and
- invalid conformance fixtures cover missing Contracts, selectors, ports, duplicate bindings, and incompatible Anchor pairs.

Failing closed is a successful test outcome when the available evidence cannot support the proposed composition.

Adding or promoting a scenario

1. Pin the exact external source revision used as evidence.

2. Document what the source actually produces and consumes.
3. Add deployable definitions according to ownership: third-party software under `DOT-Repository/DOTs/external/`, MARE-owned software under `DOT-Repository/DOTs/internal/`, and only synthetic fixtures under `DOT-Repository/DOTs/testing/`. Record their evidence and purpose in the applicable test evidence ledger and select them by exact `repository://<uuid>@<version>` identity.
4. Define Contracts only for supported, observable interactions.
5. Use contractless Anchors only when no supported Contract standard applies.
6. Declare all service ports, configuration targets, privileges, and devices.
7. Add negative fixtures for tempting but unsupported connections.
8. Start under `DOT-Testing/composition/` with validate and plan assertions.
9. Add a lifecycle dry-run before any live execution.
10. Promote to `DOT-Testing/e2e/` only with isolated resources, explicit prerequisites, reliable cleanup, and externally observable assertions.
11. Record what remains unproved in the interoperability ledger.

Interpreting results



A successful plan proves model consistency. A successful apply proves that the supported deployment adapters completed. Only an active scenario probe can prove that traffic, authentication, authorization, policy, or security behavior worked as intended.

External DOT examples

The external-product examples demonstrate how an existing open-source workload can be described as a DOT without modifying that workload. The upstream project remains authoritative for the software, configuration, and interface Contract. The DOT descriptor selects a pinned artifact and declares the portable service surface that `dotx` can validate and deploy.

Upstream documentation is useful source evidence, but it is not acceptance evidence. DOT-Design therefore tests each example at separate levels:

1. direct `dotx validate`, using the reusable publication profile where applicable;
2. Contract-selector and DOT-to-DOT Anchor composition for reusable definitions;
3. offline inspection and deployment rendering;
4. networked rendering of the exact pinned Helm charts;
5. one real Linux lifecycle on Kubernetes/Helm or Docker; and
6. bounded repeated lifecycle stress with cleanup verification.

Canonical external catalogue

The external catalogue currently has one root for each of these 17 products: APISIX, cAdvisor, Elasticsearch, Filebeat, Grafana, GraphDB, InfluxDB, Keycloak, Kibana, Loki, Eclipse Mosquitto, Prometheus node-exporter, OPA, OpenTelemetry Collector, Oxigraph, Prometheus, and Wireshark. PoC compositions reuse those identities and supply routes, realms, policies, scrape targets, data sources, credentials, files, and deployment selection as instance inputs. They do not create `pocN-*` copies of the product.

The following four products form the bounded release stress matrix. They are not the complete external catalogue.

Release stress matrix

DOT	Catalogue status	Mechanisms in the same DOT	Contract/Anchor surface	Live assertions
`Keycloak-DOT`	Stable reusable external DOT	Bitnami `HELM_CHART 25.2.0`; official Keycloak `DOCKER_COMPOSE`; official and PoC1 evidence `DOCKER_IMAGE` variants	Exact Keycloak 26.3.3 Admin REST OpenAPI plus an attributed OIDC endpoint projection; Admin realm, JWKS, and client-credentials provider Anchors	Exact StatefulSet binding and Compose `start-dev`; readiness; obtain an admin token and assert `GET /admin/realms/master` returns the master realm through both mechanisms
`Loki-DOT`	Runtime sample, not reusable	Grafana Community `HELM_CHART 18.15.1`; official Loki `DOCKER_COMPOSE` image pinned by digest	No fabricated Contract: the pinned release has HTTP documentation and internal multi-file protobuf APIs, but no supported self-contained machine-readable Contract for the exposed public HTTP surface	Exact single-binary StatefulSet binding and `/ready`; Compose `/ready`
`Prometheus-DOT`	Stable reusable external DOT	Prometheus Community `HELM_CHART 29.37.0`; official Prometheus `DOCKER_COMPOSE` image pinned by digest; `DOCKER_IMAGE` for composition-owned scrape configuration	Exact Prometheus 3.15.0 OpenAPI 3.1 artifact for build information and PromQL; explicit contractless scrape-target requirement materialized through `prometheus.yml`	Exact server Deployment binding; readiness; assert `/api/v1/status/buildinfo` reports version `3.15.0` through both mechanisms

DOT	Catalogue status	Mechanisms in the same DOT	Contract/Anchor surface	Live assertions
`Wireshark-DOT`	Runtime sample, not reusable	LinuxServer versioned `DOCKER_IMAGE`; equivalent `DOCKER_COMPOSE` image pinned by digest	No fabricated Contract: the image exposes a human web desktop, not a supported machine API	Container state/environment/volume/ports plus HTTPS for both mechanisms

Each product is kept under `DOTs/external/` because it is owned outside MARE, with its supported deployment mechanisms in the same `dot.json`. Ownership does not imply interoperability maturity. Keycloak and Prometheus publish adjacent official Contract evidence and pass the strict reusable profile. Loki and Wireshark currently provide runtime evidence only; they must not be misrepresented as Contract-backed interface interoperability. Chart versions, Compose image indexes, and the direct Wireshark image tag are deliberately pinned. `latest` is convenient for exploration but unsuitable for release evidence because a repeated run could execute different bytes. The Wireshark OCI index currently contains Linux amd64 and arm64 manifests. Stable `0.2.0` retained a historical cross-built arm64 preview, but subsequent DOT-Design release packaging and automated acceptance intentionally target Linux amd64 only.

Authoritative upstream references:

- Keycloak 26.3.3 release assets, Keycloak Admin REST API, Bitnami Keycloak chart, and Keycloak containers
- Grafana Loki Helm documentation
- Prometheus 3.15.0 official OpenAPI 3.1 artifact, Prometheus Community chart, and Prometheus installation
- LinuxServer Wireshark image

Run the examples

Run commands from the `DOT-Design` root with a `dotx` built from the checked-out `DOT-CLI` revision.

```
# Fully offline and non-mutating
task testing:dots:external:offline

# Downloads and renders the pinned charts, but does not deploy them
task testing:dots:external:dry-run

# Deploys each example once and cleans it up
task testing:dots:external:live

# Repeats the complete lifecycle three times by default
task testing:dots:external:stress
```

The live profiles require Linux, Docker, Kubernetes, Helm, `kubectl`, `curl`, `jq`, and network access to the pinned public artifacts. `dotx tool check` starts Minikube when the current environment has no selected cluster. Set `DOTX_E2E_SKIP_HELM=1` or `DOTX_E2E_SKIP_DOCKER=1` only for focused local diagnosis; a release acceptance run must not skip either runtime.

What the stress profile adds

`DOTX_E2E_STRESS_REPEATS` accepts a bounded value from 1 through 5. Every cycle uses a new namespace, Docker runtime name, Compose project names, marker, and host-port range. It executes validation again before runtime mutation, selects each mechanism explicitly, verifies workload readiness and HTTP/HTTPS reachability, and destroys all resources. The runner then fails if the cycle namespace, Wireshark container, or any of the four Compose projects still exists. If a Kubernetes deployment or readiness check fails, the runner records pod, workload, event, and bounded container-log diagnostics before cleanup and skips only the assertions that are causally blocked by that failed workload. The root failure still fails the cycle; dependent timeout noise is not counted as independent evidence.

The resulting matrix contains eight cells:

- Keycloak: `HELM_CHART` and `DOCKER_COMPOSE`;
- Loki: `HELM_CHART` and `DOCKER_COMPOSE`;
- Prometheus: `HELM_CHART` and `DOCKER_COMPOSE`; and
- Wireshark: `DOCKER_IMAGE` and `DOCKER_COMPOSE`.

This is a runtime-mechanism matrix, not a reusable-interface count. Only the Keycloak and Prometheus cells carry official machine-readable Contracts and Contract-backed Anchors. A separate locked three-DOT composition wires those two providers to a headless consumer, resolves the exact official selectors, binds the Keycloak `{realm}` parameter, and asserts the resulting operation paths and environment materializers:

```
task testing:compose:keycloak-prometheus-openapi-planning
```

```
DOTX_E2E_STRESS_REPEATS=5 task testing:dots:external:stress
```

Evidence is written to `DOT-Testing/e2e/target/external-dots/`:

- one log per cycle;
- `summary.json`, including the DOT-CLI and DOT-Repository revisions; and
- `SHA256SUMS` covering the summary and logs.

The dedicated `External DOT examples` GitHub Actions workflow runs the non-mutating profile for relevant changes. Its weekly and manually dispatched Linux job runs the stress profile and uploads the evidence bundle. Push, schedule, and manual runs use distinct concurrency lanes, so requesting a live stress run cannot cancel the required push conformance result for the same revision.

Recorded `0.2.0-rc.3` evidence

External DOT examples run 38708940363 completed two independent Linux lifecycle cycles against DOT-CLI `f8cf796afd985cc0514d42fb149ef67b6a1b7b53` and DOT-Repository `76585f6ec68a892f6cc6d48c9b2acdce6b08d48d`. Each cycle passed all 47 checks and cleaned up its resources. The complete eight-cell mechanism matrix was:

- Keycloak: Helm chart and Docker Compose;
- Loki: Helm chart and Docker Compose;
- Prometheus: Helm chart and Docker Compose; and
- Wireshark: Docker image and Docker Compose.

For both Keycloak mechanisms, the test invoked the exact Admin REST operation selected from the pinned official Contract. For both Prometheus mechanisms, it invoked the exact build-information operation selected from the pinned official Contract. The uploaded evidence contains `cycle-1.log`, `cycle-2.log`, `summary.json`, and `SHA256SUMS`; local checksum verification of the downloaded artifact passed for both cycle logs and `summary.json`. This evidence supersedes the earlier `rc.2` run because it exercises Keycloak-DOT `0.1.1` with the corrected Helm resource profile and pins the exact candidate CLI and repository revisions.

Interpreting a pass

A pass means the pinned public artifacts satisfied the assertions executed in the disposable test environment. For Keycloak and Prometheus this includes a successful request to the exact operation selected by the DOT Anchor, in addition to readiness. It does not establish production sizing, availability, security hardening, backup/restore, or every upstream feature. In particular, the Wireshark example checks its web application surface; it does not grant packet-capture capabilities, host networking, or a CDI device. Those requirements must be declared and explicitly authorized in a separate acceptance profile.

When adding another external DOT, place it under `DOTs/external/` because of its ownership and provide an immutable runtime artifact. Use a vendor/project- official Contract whenever available. Otherwise either pin a runtime-extracted Contract, maintain a clearly identified derived Contract with live conformance, or expose an adapter-owned Contract. Every Contract asset declares its authority, source revision, license, and SHA-256. A runtime-only external DOT remains valid without a `contracts/` evidence directory but makes no interoperability claim. Add production claims only after a test explicitly proves them.

Integration handoff

This handoff defines the interoperability boundary for a team implementing a catalogue, composer, Security Function Composer adapter, or other DOT consumer. It intentionally does not prescribe a service, database, API style, language, or framework.

Authoritative inputs

The reference source of reusable definitions is `DOT-Repository`. Resolve a DOT by its immutable UUID and pinned version, then verify the returned `dot.json` matches both. Resolve referenced Contract payloads without copying their protocol, schema, media, or security facts into an Anchor.

Use these commands as executable reference behavior:

```
dotx repository inspect <dot-id>@<version> --repository DOT-Repository --json
dotx validate DOT-Repository
dotx compose validate composition.yaml
dotx compose plan composition.yaml --json
```

An implementation may expose equivalent behavior through REST, gRPC, events, an SDK, or an embedded library. Its external contract should be versioned and tested independently from the DOT model.

Disposition of legacy SFC requests

The original SFC issues 6 through 9 predate the stable repository-backed model. Their consumer needs are handled at the integration boundary without creating parallel sources of truth:

- **Parameter forms:** derive an SFC-facing form from exact service-scoped environment and configuration declarations plus authoritative Contract schemas. Composition validation rejects missing or ambiguous required inputs before runtime mutation. A duplicate root `params` model is not defined.
- **Skill search:** resolve stable `Skill` UUID references through the authoritative `DOT-Repository/Skills/` catalogue. Every referenced capability has one category there; `dotx repository list --skill <uuid-or-name>` and `--category <uuid-or-name>` expose the tested consumer projection without copying names or categories into every DOT as a second controlled vocabulary.
- **Catalogue API:** `DOT-Repository` and `dotx repository` define reference behavior. A catalogue team may expose equivalent queries through any versioned API; DOT-Design does not own or prescribe that service.
- **Low-barrier authoring:** a DOT author may hand-write one `dot.json`, place its folder under the appropriate recursive `DOT-Repository/DOTs/` category, and validate the whole filesystem catalogue with `dotx validate DOT-Repository`. There is no duplicate inventory file or wrapper script. This path requires no Maven, Jsonnet, MongoDB, or running service. `dot.json` remains the sole deployment definition; a parallel YAML format and inline copies of referenced entities

are intentionally not introduced.

These dispositions settle DOT-side model ownership. They do not replace joint SFC acceptance or prove a partner catalogue/search implementation.

Required semantics

An interoperable consumer must:

- address DOTs, Contracts, and Anchors by stable UUID and pin DOT versions;
- resolve contract-backed selectors against the authoritative OpenAPI, AsyncAPI, Protocol Buffers, or GraphQL document;
- use `anchorBindings` to map a logical Anchor to a named service port;
- keep instance topology, endpoints, parameter values, credentials, and policy outside reusable DOT definitions;
- evaluate compatibility conservatively and preserve `UNKNOWN` as distinct from `COMPATIBLE`; and
- keep secret values out of DOTs, plans, persisted state, and logs.

Contractless Anchors are valid only when a supported Contract standard cannot describe the interaction. Their explicit direction, protocols, media types, and semantic types must be complementary before a wire is accepted.

Acceptance evidence

The versioned consumer profile is under `DOT-Testing/acceptance/dot-anchors/v1/`. It resolves two catalogued test DOTs, verifies the committed composition lock, and plans 21 compatible wires across all supported Contract standards and versions.

```
task testing:anchors:acceptance
task testing:conformance
```

Passing the deterministic profile proves discovery and planning semantics. A joint live test must separately prove endpoint reachability, credential and token handling, authorization, policy enforcement, and application behavior.

Future implementation freedom

The removed catalogue-service prototype is not part of the production architecture. A future team may replace it or implement discovery elsewhere in a different language or framework. Compatibility is judged by the stable DOT model and executable acceptance profile, not by reproducing the prototype's internal classes or former HTTP response shapes.

Release notes

Unreleased

Contracts now resolve only from their declaring DOT bundle or a complete digest-pinned URL declaration. The repository-level Contract catalogue and validator fallback have been removed; all remaining assets live in their owning bundles. Repository validation still rejects conflicting definitions of the same Contract UUID. The shared local/GitHub repository gate additionally validates every bundle in isolation and offline.

Getting started now begins with a checksummed Linux-amd64 CLI installation and a complete Prometheus Docker Compose lifecycle, with API/version verification, cleanup, and troubleshooting. Contributor toolchains and shared CI gates have their own guide. Antora includes the same command blocks executed by `testing:quickstart:dry-run` and `testing:quickstart:live`; the external-DOT workflow rehearses both, including port overrides and cleanup after failure. Development pages now export their own PDF alongside the latest stable PDF, matching the existing download link in the site header.

The Antora site now includes a guided demo for presentations and onboarding, with a tested five-minute read-only path, expected evidence, an optional live Contract-probe path, and explicit support boundaries.

`dotx compose apply --dry-run` no longer creates or changes the composition state/lock directory. Lifecycle rendering therefore works with a read-only cache while retaining the same fail-closed planning and runtime checks.

0.2.4

`0.2.4` is a backward-compatible Linux-amd64 hardening release for the stable `dot.mare6g.eu/v1` single-host composition profile. It preserves the DOT, Contract, Anchor, composition-schema, repository-layout, deployment, and CLI interfaces published in `0.2.3`. The exact pinned revisions are DOT-CLI `e88daa83221ae5d49a1317b7412045ab2b78315f` and DOT-Repository `247606d4a55c5e85a822f1f82eca52a3574b5a04`.

DOT-CLI now exposes a local/GitHub `task ci:security` gate using the pinned stable OSV-Scanner release. It checks the module dependency graph and performs Go call analysis against the current OSV database. The network-backed scan is kept separate from deterministic `task ci` because vulnerability data changes independently of a source revision.

Standalone DOT-CLI packaging now emits a deterministic CycloneDX 1.6 SBOM for the exact Linux-amd64, CGO-disabled build profile. Release tests validate the SBOM identity, platform properties, dependency content, omitted timestamp and serial number, checksum coverage, and byte-for-byte equality across two package builds. Downloaded-binary acceptance retains the published SBOM as evidence.

Readable workflow badge publication now refreshes the complete status set on a coalesced 15-minute schedule, with manual dispatch available for immediate refresh. It no longer consumes one self-hosted-runner job for every completed workflow in the already large validation matrix.

Wasmtime lifecycle status and inventory checks now tolerate the short, bounded interval in which Linux procfs may expose an empty process command line during `exec`. A non-empty command line that does not match the recorded Wasmtime module identity is still rejected immediately, preventing lifecycle operations from acting on a reused PID.

Antora publication now deduplicates an exact stable tag from its identical release-commit worktree, closing the tag-push race while preserving immutable stable documentation. Current-profile documentation is guarded against the machine-readable acceptance profile: twenty-one wires, all four finite side-effect-free gRPC shapes, and MQTT, NATS, Kafka, and AMQP 0-9-1 delivery.

The complete pushed workflow matrix passed on `77d8f099e5f42430f1d30e8d01c27b2c534b8d49`. It includes 100% DOT-Domain instruction and branch coverage, 100% DOT-CLI aggregate and per-executable- package statement coverage, repository conformance, all twenty-one cross-standard wires, real-producer and real-broker active probes, Docker, Kubernetes, Helm, Vagrant, and WebAssembly lifecycle evidence, and versioned Antora publication. DOT-CLI `v0.2.4` additionally passed two-pass reproducible packaging and downloaded-binary acceptance.

The release publishes the versioned DOT-Domain JAR, DOT and Composition v1 JSON Schemas, JSON and XML CycloneDX SBOMs, the Linux-amd64 `dotx` binary, an exact source manifest, and checksums. Linux amd64 remains the only supported release target. DOT-Registry REST API work, native arm64, non-Linux targets, multi-host orchestration, and joint Security Function Composer or partner acceptance remain outside this release boundary.

0.2.3

`0.2.3` is a backward-compatible production-hardening release for the stable `dot.mare6g.eu/v1` single-host composition profile. It preserves the DOT, Contract, Anchor, composition-schema, repository-layout, and deployment interfaces published in `0.2.2` while broadening standards-authorized runtime evidence and strengthening deployment-configuration acceptance. The exact pinned revisions are DOT-CLI `b07ec3b568b59cb25631a9b51700a14ee124bb05` and DOT-Repository `247606d4a55c5e85a822f1f82eca52a3574b5a04`.

The cross-runtime configuration matrix now exercises every executable adapter through separate default-only and explicit-override paths. It verifies non-sensitive `defaultValue` and `defaultContent` materialization, explicit environment/file precedence, resolved service targeting, and redacted dry-run diagnostics.

Contract-authorized gRPC probes now cover unary, server-streaming, client-streaming, and bidirectional methods. All methods still require the Proto3 `NO_SIDE_EFFECTS` option and server reflection. Streaming fixtures must declare an exact finite response count; input and output messages are validated against the authoritative descriptors; and expectations explicitly select `ALL`, `ANY`, `FIRST`, or `LAST` response matching.

AsyncAPI delivery now supports NATS core messaging beside MQTT. NATS fixtures require the official channel binding and a concrete non-wildcard subject, then prove UUID-correlated subscribe-before-publish delivery through digest-pinned official NATS client and server images. The live standards fixture exercises all four gRPC shapes plus MQTT and NATS while retaining redacted state evidence.

AsyncAPI delivery also supports Kafka through the official 0.5.0 channel binding. `dotx` honors a binding-level `topic` override, rejects invalid Kafka topic names and MQTT-only QoS, waits for a correlation-scoped consumer-group assignment before publishing, and verifies the exact correlated record through the digest-pinned official Apache Kafka 4.3.1 JVM image. The ephemeral consumer group is removed; the record itself remains subject to the topic's broker retention policy. The canonical runtime-only Kafka DOT expands the locked standards composition to twenty wires and three real message brokers.

AsyncAPI delivery now also supports AMQP 0-9-1 through the official 0.3.0 binding. `dotx` validates exact exchange, routing-key, virtual-host, durability, auto-delete, and delivery-mode semantics; binds an exclusive auto-delete probe queue before publish; acknowledges and correlation-matches the delivered JSON; and obtains RabbitMQ credentials only through a secret reference. Queue-bound channels, headers exchanges, AMQPS, and AMQP 1.0 fail closed. The canonical digest-pinned RabbitMQ 4.3.6 runtime DOT expands the locked standards composition to twenty-one wires and four real message brokers.

The complete pushed workflow matrix passed on `79807aacebe7aab1112fe82104898ab4a0929db0`, including DOT-CLI CI, Contract active probes, compatibility matrix, coverage, Kubernetes and Helm runtime E2E, DOT repository conformance, and versioned Antora publication.

The release publishes the versioned domain JAR, DOT and Composition v1 schemas, CycloneDX JSON/XML SBOMs, Linux-amd64 `dotx` binary, immutable release manifest, and checksums. The tag workflow performs deterministic two-build packaging before publication, and post-publication acceptance downloads and exercises the released binary.

Existing `0.2.0` through `0.2.2` definitions, compositions, and locks remain valid. Unsafe or unbounded gRPC, AsyncAPI transports other than MQTT, NATS, Kafka, and AMQP 0-9-1, AMQPS/AMQP 1.0, multi-scheme authentication, multi-host orchestration, automatic cross-platform gateway provisioning, general stateful-data migration, native arm64, non-Linux executables, DOT-Registry REST API work, and joint SFC/partner acceptance remain outside this Linux-amd64 release.

0.2.2

`0.2.2` is a backward-compatible extension of the stable `dot.mare6g.eu/v1` single-host composition profile. It retains the DOT, Contract, Anchor, composition-schema, repository-layout, and deployment interfaces from `0.2.1` while completing the standards-authorized active-probe profile. The exact pinned revisions are DOT-CLI `2dafb492bb657da4d6e6dfe3eab4e9ee4ee229c9` and DOT-Repository `31357e57ab1332e93de6afc25a2b0d67aafed857`.

GraphQL query probes validate documents, variables, and optional exact response expectations against the authoritative SDL. Mutations require verification and cleanup operations; GraphQL-over-SSE subscriptions require a separately triggered event plus cleanup. Unary gRPC probes require server reflection, protobuf-JSON request validation, and the Proto3 `NO_SIDE_EFFECTS` option. AsyncAPI MQTT probes require an explicit binding, concrete topic, JSON-Schema-valid correlated message, QoS, subscribe-before-publish ordering, bounded timeout, and exact non-retained delivery. All fixtures remain composition-owned and persisted evidence remains redacted.

The fifteen-wire deterministic matrix covers OpenAPI 3.0/3.1/3.2, AsyncAPI 3.0/3.1, Proto3, and

GraphQL. A real GraphQL.js/gRPC producer and Eclipse Mosquitto broker provide live Docker acceptance. The complete pushed workflow matrix passed on `70b0e94760d60bc0fc37f887db0466796627829a`, including Contract active probes, coverage, compatibility, and Antora publication.

Existing `0.2.0` and `0.2.1` definitions and compositions remain valid; probe fields are optional and additive. Streaming or safety-unclassified gRPC, non-MQTT AsyncAPI transports, multi-scheme probe authentication, native arm64, non-Linux executables, multi-host orchestration, DOT-Registry REST API work, and joint SFC/partner acceptance remain outside this Linux-amd64 release.

0.2.1

`0.2.1` is a backward-compatible hardening release for the stable `dot.mare6g.eu/v1` single-host composition profile. It retains the `0.2.0` DOT, Contract, Anchor, composition-schema, repository, and deployment interfaces. The exact pinned revisions are DOT-CLI `82d7de45800f4e4a435ef08202d4f1e42c16eca8` and DOT-Repository `2fe0ecc81f9caea43c5bf6e0dc824c3e0faf34e3`.

The patch adds overflow-safe SemVer precedence, atomic WASM module-cache replacement and restart-profile verification, fail-closed empty Docker port forwarding, correct Minikube home-directory expansion, and child-process checks for detached Kubernetes forwarding. Repository commands now share one verified private-cache acquisition path. DOT-Domain enforces 100% instruction and branch coverage, while DOT-CLI enforces 100% aggregate and per-executable-package statement coverage. The installer missing-`curl` negative test is hermetic and cannot fall through to host networking or `sudo`.

The complete 13-workflow matrix passed on the pushed hardening baseline `7c71e60393b1758d45e3558a15a2073450cb5cfc`, including coverage, Anchor conformance, and repository conformance. The tag workflow must also pass deterministic two-build packaging and checksum verification. Post-publication acceptance downloads and exercises the released binary rather than trusting only source-tree tests.

The release publishes the versioned domain JAR, DOT and Composition v1 schemas, CycloneDX JSON/XML SBOMs, release manifest, checksums, and `dotx` for Linux amd64. Linux amd64 is the only supported executable target. DOT-Registry REST API work, native arm64, non-Linux targets, and joint Security Function Composer/partner acceptance remain outside this release boundary.

0.2.0

`0.2.0` promotes the fully accepted `0.2.0-rc.8` candidate as the first stable release of the `dot.mare6g.eu/v1` composition profile. There are no behavioral changes from that candidate: the stable tag regenerates the versioned domain artifact, schemas, SBOMs, documentation, and Linux CLI assets from the same pinned DOT-CLI `4f415d43f08bb546d333ab61d6de272e92113903` and DOT-Repository `2fe0ecc81f9caea43c5bf6e0dc824c3e0faf34e3` revisions.

The release pipeline classifies `0.2.0` as stable and publishes it as the latest GitHub release. Before promotion, `0.2.0-rc.8` passed the full pushed commit matrix, reproducible release packaging, and post-publication Linux amd64 acceptance using the downloaded checksummed binary, including

two repeated external-DOT lifecycle and cleanup cycles. Arm64 remains a cross-built preview until equivalent native evidence exists, and joint SFC acceptance remains a separate integration activity.

0.2.0-rc.8

`0.2.0-rc.8` supersedes `0.2.0-rc.7` with a current stable build and CI toolchain while retaining the accepted `dot.mare6g.eu/v1` composition, runtime, lifecycle, Contract, Skill-taxonomy, reusable-catalogue, and WASI profiles. Node.js 26 is now the explicit documentation and Contract-validation baseline; Go 1.27.2, Rust 1.98.1, Ruby 4.0, Java 27, and Maven 3.10.0 provide the remaining release toolchains. Direct Maven and Go dependencies and all third-party GitHub Actions were updated to compatible stable releases and immutable Action SHAs, with weekly Dependabot checks for Actions, Maven, Go, and the independently cloneable repositories. Documentation CI now rejects prerelease Ruby runtimes before Antora/PDF assembly.

Release publication now uses a tested, fail-closed SemVer channel classifier: prerelease versions such as `0.2.0-rc.8` are published as GitHub prereleases, while `0.2.0` is published as the latest stable release. The exact pinned revisions are DOT-CLI `4f415d43f08bb546d333ab61d6de272e92113903` and DOT-Repository `2fe0ecc81f9caea43c5bf6e0dc824c3e0faf34e3`.

0.2.0-rc.7

`0.2.0-rc.7` supersedes `0.2.0-rc.6` with two release-evidence hardening changes while retaining the same stable `dot.mare6g.eu/v1` profile and pinned DOT-CLI and DOT-Repository revisions. Source-adapted PoC harnesses now acquire private producer source through one guarded helper: it checks out the full pinned commit in a fresh temporary directory, verifies `HEAD` and the expected GitHub origin, retries bounded transient failures, and rejects partial, untrusted, missing-revision, or unsafe-destination results. Deterministic tests exercise those positive and negative boundaries without network access.

Contract conformance now installs the official Protocol Buffers 36.2 compiler for Linux amd64 or arm64 through the shared local bootstrap, verifies the download with an architecture-specific SHA-256, and uses that exact binary for both positive and deliberately invalid Protobuf fixtures. Local Taskfile runs and GitHub Actions no longer depend on different compiler setup paths. The exact pinned revisions remain DOT-CLI `b01f736393caa1a93d32e2c5593e032e854052ac` and DOT-Repository `d0b7fc1f3aac13a2b785482b07d5529f1d53404f`.

0.2.0-rc.6

`0.2.0-rc.6` supersedes `0.2.0-rc.5` with one shared, fail-closed bootstrap for the official WASI HTTP fixture. Local `task ci:live` and GitHub Actions now fetch and build the same exact Bytecode Alliance `sample-wasi-http-rust` revision before executing the same Wasmtime lifecycle assertions. The stable `dot.mare6g.eu/v1` composition profile, runtime adapters, lifecycle behavior, Contract model, Skill taxonomy, and reusable catalogue structure are otherwise unchanged. The exact pinned revisions remain DOT-CLI `b01f736393caa1a93d32e2c5593e032e854052ac` and DOT-Repository `d0b7fc1f3aac13a2b785482b07d5529f1d53404f`.

The bootstrap verifies the exact upstream URL and commit, rejects non-Git, wrong-origin, and

modified tracked source caches, and rebuilds the `.wasm` component rather than trusting an ignored build artifact. The GitHub workflow no longer contains a separate checkout/build implementation; it invokes the same `ci:wasm-runtime` Taskfile gate available locally. Documentation records the Rust 1.98.1, `wasm32-wasip2`, and first-build network prerequisites and the optional `WASI_HTTP_COMPONENT` override for a deliberately supplied component.

0.2.0-rc.5

`0.2.0-rc.5` supersedes `0.2.0-rc.4` with one fail-closed representation for complete Contract payloads. The stable `dot.mare6g.eu/v1` composition profile, runtime adapters, lifecycle behavior, Skill taxonomy, and reusable catalogue structure are unchanged. The exact pinned revisions are DOT-CLI `b01f736393caa1a93d32e2c5593e032e854052ac` and DOT-Repository `d0b7fc1f3aac13a2b785482b07d5529f1d53404f`.

Complete Contract declarations now carry a structured `payload` object:

```
"payload": {
  "url": "https://vendor.example/contracts/service-openapi.yaml",
  "sha256": "0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef"
}
```

The URL must be absolute HTTPS without user information or a fragment, and the digest must be exactly 64 lowercase hexadecimal characters. Raw embedded Contract documents and the former flat `payloadUrl`/`payloadSha256` fields now fail schema and `dotx validate` checks. UUID-only Contract references remain a migration-compatible form for verified repository-local wrapper definitions.

`dotx validate` downloads the authoritative bytes, verifies their digest, and uses `MARE_READ_ONLY_TOKEN` or `GITHUB_TOKEN` only for `raw.githubusercontent.com`. Authorization is removed before following a cross-host redirect. Unit and repository tests cover malformed URLs and digests, unavailable assets, digest mismatch, authentication scoping, redirect credential leakage, inline payload rejection, flat-field rejection, and the official external Contract composition. The standalone DOT-Repository gate also runs the pinned validator directly for every catalogue revision.

GitHub Actions and local pre-push verification now share root `ci:*` Taskfile gates. `task ci:deterministic` covers the deterministic non-live release boundary; `task ci:live` covers the mutable Linux runtime workflows and requires the same official WASI HTTP component exercised by Actions, preventing a reduced local test path from being mistaken for workflow parity.

0.2.0-rc.4

`0.2.0-rc.4` retains the runtime and lifecycle profile accepted in `0.2.0-rc.3` and completes the closed SFC-requirement audit with repository-backed Skill discovery. `DOT-Repository/Skills/` now contains an authoritative asset for every capability UUID referenced by the catalogue and links each capability to one of the five controlled MARE categories.

`dotx validate DOT-Repository` resolves all 67 DOT definitions against that catalogue and rejects missing capabilities, category assets used as capabilities, duplicate references, and invalid category

links. `dotx repository list --skill <uuid-or-name>` and `--category <uuid-or-name>` provide resolved, machine-readable discovery without copying category fields into every `dot.json`. The exact published revisions were DOT-CLI `3069f907bfd2056b16b92800bb5b98552c28832d` and DOT-Repository `3117a18da3004c57f4edf3714494fd3954225ae7`.

The audit also reconfirmed the existing DOT-side dispositions for stable Anchor identity and semantics, authoritative payload Contracts, service-scoped pre-deployment configuration validation, and single-file `dot.json` authoring. The separate DOT-Registry REST API remains outside this candidate, and joint SFC/partner acceptance remains a manual gate.

External products now have one canonical reusable DOT root rather than PoC-specific identities. Seventeen product roots plus two shared external test fixtures replace the former duplicated Keycloak, OPA, Prometheus, Elasticsearch, and APISIX definitions. PoC realms, policies, routes, scrape targets, data sources, secrets, and files remain composition-owned inputs; unavoidable source adaptations are named deployment variants under the same product UUID. Repository validation rejects any new deployable definition below `DOTs/external/pocs/`.

Composition lifecycle commands preserve the exact deployment ID rather than reducing it to a manifest type. The `--deployment` selector prefers an exact ID and accepts a manifest type or alias only when it identifies one deployment, so source-adapted and upstream deployments of the same type cannot be selected silently.

Catalogue placement now records ownership rather than Contract maturity: third-party software is under `DOTs/external/`, MARE-owned/developed software is under `DOTs/internal/`, and only synthetic fixtures remain under `DOTs/testing/`. Every DOT bundle includes a local quickstart `README.md`. External/internal bundles opt into strict reusable validation by publishing a `contracts/` evidence directory whose assets identify their authority as upstream-official, runtime-extracted, DOT-ecosystem-derived, or adapter-owned, along with source revision, license, and SHA-256. Loki and Wireshark are now correctly classified as external runtime-only definitions without fabricated vendor Contracts.

0.2.0-rc.3

`0.2.0-rc.3` supersedes `0.2.0-rc.2` after post-publication, exact-pin lifecycle acceptance found that the Keycloak Helm workload could not reliably complete Quarkus augmentation within its declared 1536-MiB container limit. Three independent real-cluster runs retained pod and event diagnostics showing `OOMKilled` exit 137; the runner had 62 GiB of host memory and Minikube exposed 16.8 GB and 20 CPUs, isolating the defect to the DOT resource declaration.

Keycloak-DOT `0.1.1` declares a 2-GiB memory request and 4-GiB limit while retaining the same official Keycloak 26.3.3 Contract, Anchor selectors, chart, image, service, and Docker Compose mechanism. The external-DOT render suite now inspects the real upstream chart output and rejects any mismatch in the Keycloak container's CPU or memory requests and limits before live deployment. The official external-Contract composition and content-addressed lock resolve the versioned `0.1.1` definition.

External DOT examples run 38708940363 passed both the non-mutating chart conformance job and two complete Linux lifecycle cycles. Each cycle passed all 47 checks with zero failures and cleanup verification across Keycloak, Loki, Prometheus, and Wireshark's eight deployment-mechanism

cells. The downloaded, checksum-verified evidence pins DOT-CLI `f8cf796afd985cc0514d42fb149ef67b6a1b7b53` and DOT-Repository `76585f6ec68a892f6cc6d48c9b2acdce6b08d48d`.

This candidate also adds a post-publication workflow that downloads the actual checksummed Linux amd64 `dotx` release asset, verifies its release manifest and source revisions, then runs repository validation, Anchor acceptance, Composition v1 acceptance, and two complete external-DOT lifecycle cycles. Maven dependency submission is now an explicit self-hosted workflow; the billing-blocked GitHub-managed automatic submission feature is disabled. Linux amd64 is the release-accepted executable target. The cross-built arm64 asset remains preview-only until equivalent native arm64 evidence exists. Joint SFC/partner acceptance remains a separate manual gate and is not claimed by this candidate.

0.2.0-rc.2

`0.2.0-rc.2` supersedes the first candidate with the fixes and acceptance evidence completed after `0.2.0-rc.1` was tagged. The stable serialized composition API remains `dot.mare6g.eu/v1`.

Changes since rc.1

- Exact service-to-workload selection and configuration isolation for multi-service Docker Compose, Kubernetes-manifest, and Helm deployments.
- Exact official, digest-pinned OpenAPI Contracts and Contract-backed Anchors for reusable Keycloak and Prometheus external DOTs, with every Helm and Docker Compose deployment binding the same interface.
- `dotx validate --profile reusable` now enforces declared HTTPS Contract provenance, payload integrity, selector resolution, Contract use, and per- deployment Anchor binding. Review establishes that the pinned source is the official upstream artifact. `dotx validate <repository-root>` recursively discovers every `dot.json`; the later rc.4 ownership migration derives strict reusable validation from adjacent Contract evidence rather than category path, and is the single DOT-Repository GitHub Actions gate; no duplicate inventory or shell wrapper is required.
- Same-DOT alternative deployment mechanisms for the Keycloak, Loki, Prometheus, and Wireshark external-product examples. Loki and Wireshark were initially retained as runtime-tested samples; rc.4 reclassifies them as external runtime-only DOTs without claiming Contract-backed interoperability.
- A locked Keycloak/Prometheus/consumer composition proves that both stable external Anchors resolve and materialize the exact official operations.
- Failed Kubernetes/Helm workloads now retain pod, event, workload, and bounded log diagnostics before cleanup; causally blocked endpoint assertions are skipped rather than counted as independent failures.
- Repeated hosted-Linux acceptance run 38600327954 covering all eight external-DOT mechanism cells: two complete lifecycle cycles, 46 checks per cycle, zero failures, verified cleanup, and four successful official- Contract operation checks per cycle. The checksummed evidence pins DOT-CLI `848b2d56e9e416ae87fa46d221c9649041b64541` and DOT-Repository `b681c937422fa3db13ad16587cf271700eff5d79`.

- Portable evidence checksums and retained GitHub Actions artifacts.
- Separate workflow concurrency lanes for push conformance and manually requested live stress, preventing one evidence class from canceling the other.

Linux amd64 is the release-accepted executable target. The published arm64 executable is a cross-built preview and is not a supported final-release target until equivalent native arm64 evidence exists. Producer/partner acceptance, non-Linux executables, multi-host orchestration, general stateful migration, and automatic provisioning of cross-platform gateways remain outside this candidate. GraphQL, gRPC, and event-driven runtime probes beyond the documented safe profile remain standards-led roadmap work rather than an `rc.2` blocker.

The candidate assets and verification procedure are the same classes described for `rc.1` below, with filenames versioned as `0.2.0-rc.2`. The tag workflow rebuilds the Java artifact, both schemas, SBOMs, and Linux `dotx` binaries from the immutable revisions recorded in `release-manifest.json`, then verifies reproducibility and `SHA256SUMS` before publication.

0.2.0-rc.1

`0.2.0-rc.1` is the first integrated release candidate for the stable `dot.mare6g.eu/v1` composition profile. It replaces the narrower 0.1.0 artifact set with a traceable domain model, both machine-readable schemas, and version-stamped `dotx` binaries built from the exact DOT-CLI revision pinned by this repository.

Highlights

- Stable, repository-backed Composition v1 lifecycle, probe, and upgrade commands.
- Contract-driven Anchors for OpenAPI 3.0/3.1/3.2, AsyncAPI 3.0/3.1, Protobuf/gRPC, GraphQL, and declared contractless interactions.
- Immutable repository resolution, content-addressed locks, resumable lifecycle operations, and last-known-good upgrade rollback.
- Managed Docker image, Docker Compose, Kubernetes manifest, Helm chart, Vagrant/libvirt, and Wasmtime/WASI profiles.
- Explicit cross-platform networking through externally owned gateway bindings with pinned host, port, ownership, and no-cleanup semantics.
- Service-scoped configuration, secrets, OCI requirements, CDI devices, and exact native-workload bindings.

Assets and verification

The candidate includes the DOT-Domain JAR, DOT and Composition v1 schemas, CycloneDX SBOMs, and `dotx` binaries for Linux amd64 and arm64. `release-manifest.json` records the immutable DOT-Design, DOT-CLI, and DOT-Repository revisions. The repository revision pins its `DOTs/`, `Skills/`, and `Schema/` assets together. Verify the download set with:

```
sha256sum --check SHA256SUMS
```

The tag workflow publishes only after the Java build, generated-schema drift check, DOT-CLI tests, repository test, Composition v1 acceptance suite, deterministic packaging comparison, manifest validation, and checksum verification pass.

Compatibility and boundaries

The serialized composition API is `dot.mare6g.eu/v1`; breaking serialized changes require a new API version. The experimental local catalogue/registry prototype from the 0.1.0 era is not included: DOT-Repository is authoritative.

This candidate does not claim multi-host orchestration, automatic provisioning of cross-platform gateways/tunnels/brokers, general stateful data migration, arbitrary privileged workloads, non-Linux `dotx` executables, or producer/partner acceptance. The current CLI includes Linux process controls for its managed WASI runtime, so this candidate publishes Linux amd64 and arm64 binaries only. Closed legacy SFC issues record DOT-side disposition; joint consumer acceptance remains a separate manual gate.

See the repository-root `RELEASE_NOTES.md` included with the source for the complete candidate notes and upgrade guidance.

DOT-Domain

Domain model

`DOT-Domain` is the persistence-neutral Java library that defines the portable DOT aggregate and its validation rules. It contains no application, web, or storage-adapter layer.

Aggregate overview

Type	Responsibility
`Dot`	Aggregate root: identity, version, description, classification groups, metadata, Skills, Contracts, Anchors, and Runtimes.
`Skill`	Standalone human-readable capability description.
`Contract`	Standalone authoritative OpenAPI, AsyncAPI, Protobuf, or GraphQL document.
`Anchor`	Embedded stable logical point through which the DOT participates in composition.
`Runtime`	A target `Platform` plus one or more compatible deployment definitions.
`Deployment`	Stable deployment ID, one polymorphic manifest, and the logical services it materializes.
`DeploymentService`	Service identity, host, named ports, Contract references, Anchor bindings, and its supported configuration surface.

`Dot`, `Skill`, and `Contract` are independently identified domain entities. Other types are embedded values owned by a DOT. The module carries no database or ORM annotations; persistence is an adapter concern.

Identity, versioning, and references

Root entity IDs are UUIDs. Equality for `Dot`, `Skill`, and `Contract` is ID-based so changes to mutable metadata or collections do not destabilize sets or persistence identity.

`Version` accepts Semantic Versioning 2.0.0 values and the special `latest` selector. It serializes as a string and implements SemVer precedence; `latest` sorts after concrete versions. A new `Dot` or `Contract` defaults to `0.0.0`, while `Platform.version` defaults to `latest` so no platform-specific release is implied.

JSON serializes `Dot.skills`, `Dot.contracts`, and `DeploymentService.contracts` as ID-only objects. Consumers resolve those UUIDs against authoritative definitions rather than relying on object embedding.

Catalogue uniqueness rules are:

- `Skill.name` is globally unique;

- `Dot` is unique by `(name, version)`; and
- `Contract` is unique by `(name, version)`.

Contracts

Standard	Payload format	Anchor selector
`OPENAPI`	`YAML` or `JSON`	RFC 6901 URI-fragment JSON Pointer to an Operation Object
`ASYNCAPI`	`YAML` or `JSON`	RFC 6901 URI-fragment JSON Pointer to an Operation Object
`PROTOBUF`	`PROTOBUF`	Fully qualified service method
`GRAPHQL`	`GRAPHQL`	Root-field schema coordinate

`Contract.isFormatCompatibleWithStandard()` rejects mismatched standard/format pairs. Payload parsing and selected-element existence are resolver concerns; the domain model validates selector syntax and references within the aggregate.

Anchors and bindings

A contract-backed Anchor contains:

- a stable UUID and display name;
- a `PROVIDES` or `REQUIRES` role;
- optional semantic-type URIs; and
- a Contract UUID plus standard-specific selector.

It omits direction, protocol, and media type because those facts come from the selected Contract operation. A contractless Anchor must declare `direction`, at least one protocol, and at least one media type explicitly.

`DeploymentService.anchorBindings` maps an Anchor UUID to a named `ServicePort` on that same service. An Anchor contains no host, numeric port, credential, placement, or live policy state.

The class-level `@ValidAnchor` constraint delegates cross-reference validation to `AnchorValidator`. It rejects:

- duplicate Anchor UUIDs within one DOT;
- Contract references not declared by the same DOT;
- selectors invalid for the referenced Contract standard;
- bindings to undeclared Anchors;
- bindings to port names absent from that service; and
- duplicate bindings of one Anchor within the same service.

See DOT Anchors and Composition for authoring and wiring examples.

Runtime two-layer model

`Runtime` separates infrastructure from deployment mechanism:

Enum	Used by	Answers
`PlatformEngine`	`Platform.engine`	Where the workload runs: `DOCKER`, `KUBERNETES`, `VAGRANT`, or `WEBASSEMBLY`.
`DeploymentManifestType`	`DeploymentManifest.type`	How it is deployed: Docker image, Docker Compose, Kubernetes manifest, Helm chart, Vagrantfile, or WASM module.

Every `DeploymentManifestType` declares its required `PlatformEngine`. `Runtime.isDeploymentPlatformAligned()` validates every deployment against the runtime platform. `Runtime.isDeploymentIdsUnique()` enforces stable deployment selection within that runtime.

`Deployment.services` is a set. `DeploymentService` uses ID-based equality, so logical service IDs remain unique within the deployment.

Deployment manifests

`DeploymentManifest` is polymorphic JSON dispatched by its existing `type` property.

Type	Java subtype	Important fields
`DOCKER_IMAGE`	`DockerImage`	Image reference and ordered dotenv payloads
`DOCKER_COMPOSE`	`DockerCompose`	Complete YAML/JSON payload and ordered dotenv payloads
`KUBERNETES_MANIFEST`	`KubernetesManifest`	Complete YAML/JSON manifest payload
`HELM_CHART`	`HelmChart`	Repository, chart or OCI reference, version, and ordered values payloads
`VAGRANTFILE`	`Vagrantfile`	Complete Ruby DSL payload
`WASM_MODULE`	`WebAssemblyModule`	Relative/file or HTTPS module URI, WASI execution world, command arguments, and required SHA-256 integrity pin

Docker Compose and Kubernetes manifest payloads accept YAML or JSON only. Helm values also preserve YAML/JSON order: later entries override earlier entries as with repeated Helm `-f` arguments.

Runtime instance concerns do not belong in a manifest. For example, a Helm release name and namespace are selected by deployment tooling, not embedded in `HelmChart`.

Service configuration declarations

Every `DeploymentService` owns one `DeploymentConfig` containing:

- environment variables, including default, required, and sensitive metadata;
- configuration-file paths, content type, defaults, and sensitivity;
- logical volume mounts;
- logical CNCF CDI device requirements; and
- optional OCI capability and host-namespace requirements.

These are supported inputs and requirements, not applied runtime state. Operator values, host paths, concrete CDI device names, and permission grants belong to deployment or composition input.

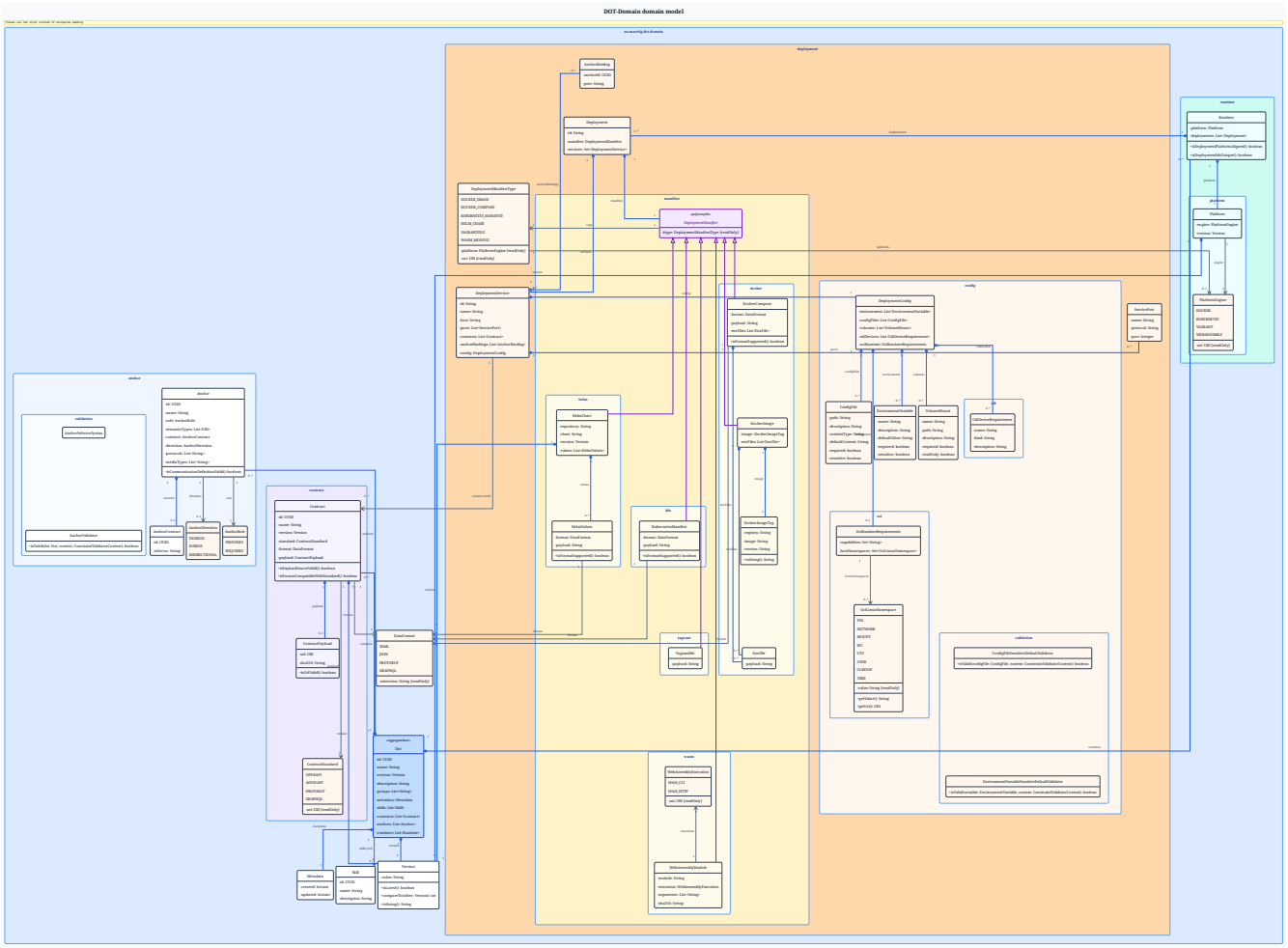
OCI requirements use standard `CAP_*` capability names and typed Linux namespaces. The model deliberately has no unrestricted `privileged` flag.

A CDI requirement declares a stable logical name and a `vendor.example/class` kind. A composition binds it to one exact `vendor.example/class=device` identifier. CDI installation, allocation, and authorization remain operator responsibilities.

Serialization and persistence adapters

- Collections have initialized builder defaults and do not become `null` when constructed through Lombok builders.
- `Version` is JSON-serialized as a normalized string.
- Persistence adapters must preserve UUID and version representations.
- Referenced `Skill` and `Contract` objects expose only `id` when nested in DOT JSON.
- Domain values carry no database or ORM annotations.

Class diagram



The diagram is generated from current domain sources during the `DOT-Domain` build. Open it separately for the full-resolution view.

Build and test

```
# Unit tests only; no service or database required
mvn -pl DOT-Domain test

# Regenerate schema and diagram through the reactor lifecycle
mvn -pl DOT-Domain clean install
```

Representative tests cover version precedence, reference serialization, manifest polymorphism, runtime/manifest alignment, deployment configuration, Anchor rules, and entity equality.

Related pages

- Architecture
- DOT Anchors and Composition
- Integration handoff